Česká zemědělská univerzita v Praze

Provozně ekonomická fakulta

Katedra informačních technologií

Diplomová práce

Implementace GitOps pro správu cloudové infrastruktury

Pavel Dikan

© 2026 ČZU v Praze

ČESKÁ ZEMĚDĚLSKÁ UNIVERZITA V PRAZE
Provozně ekonomická fakulta

ZADÁNÍ DIPLOMOVÉ PRÁCE

Pavel Dikan

Informatika

Název práce:
Implementace GitOps pro správu cloudové infrastruktury

Název anglicky:
GitOps implementation for cloud infrastructure management

Cíle práce
Analýza přístupů k implementaci GitOps pro efektivní správu cloudové infrastruktury a její automatizaci.

Úkoly výzkumu:

1. Prozkoumat koncept GitOps a jeho roli v řízení infrastruktury.
2. Porovnat GitOps s tradičními metodami správy cloudové infrastruktury.
3. Analyzovat aktuální nástroje podporující GitOps (ArgoCD, Flux, Terraform, Helm).
4. Popsat proces implementace GitOps v cloudovém prostředí.
5. Zhodnotit výhody, nevýhody a best practices GitOps v podnikovém nasazení.

Metodika (krátká verze pro zadání)
Bude provedeno podrobné studium teoretických základů konceptu GitOps a jeho uplatnění při správě cloudové infrastruktury. To bude zahrnovat porovnání s tradičními metodami, přehled dostupných nástrojů a jejich vlastností. Na základě analýzy bude navržen modelový přístup k implementaci GitOps v cloudovém prostředí. Tento návrh bude sloužit jako základ pro doporučení osvědčených postupů vhodných pro podnikové využití.

Doporučený rozsah práce

60-80s.

Klíčová slova

monitoring, infrastruktura, implementace, konfigurace, síť, server, síťový provoz, software

---

Doporučené zdroje informací:

1. BUELTA, Jaime. Using GitOps Principles. In: Hands-On Docker for Microservices with Python. United Kingdom: Packt Publishing, Limited, 2019. ISBN 1838823816;9781838823818
2. JOYNER, John a Steve BUCHANAN. GitOps Insights. In: Azure Arc-Enabled Kubernetes and Servers. United States: Apress L. P, 2021, s. 255–265. ISBN 9781484277676;1484277678
3. MISHRA, Pravin. GitOps: The Future of Operations. In: Ultimate Git and GitHub for Modern Software Development. India: Orange Education PVT Ltd, 2024
4. SALECHA, Rohit. Practical GitOps: Infrastructure Management Using Terraform, AWS, and GitHub Actions. 1. vyd. Berkeley, CA: Apress, 2022;2023; ISBN 9781484286739;1484286731;1484286723;9781484286722;9781480000000
5. VINTO, Natale a Alex SOTO BUENO. GitOps cookbook: Kubernetes automation in practice. First. vyd. Beijing;Farnham;Sebastopol;Tokyo;Boston;: O'Reilly, 2023. ISBN 9781492097471;1492097470

Předběžný termín obhajoby
2025/26 LS – PEF

Vedoucí práce
Ing. Martin Havránek, Ph.D.

Garantující pracoviště
Katedra informačních technologií

Elektronicky schváleno dne 4. 9. 2026

doc. Ing. Jiří Vaněk, Ph.D.
Vedoucí katedry

Elektronicky schváleno dne 3. 11. 2026

doc. Ing. Tomáš Šubrt, Ph.D.
Děkan

V Praze dne 26. 03. 2026

Čestné prohlášení

Prohlašuji, že svou diplomovou práci " Implementace GitOps pro správu cloudové infrastruktury" jsem vypracoval samostatně pod vedením vedoucího diplomové práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou citovány v práci a uvedeny v seznamu použitých zdrojů na konci práce. Jako autor uvedené diplomové práce dále prohlašuji, že jsem v souvislosti s jejím vytvořením neporušil autorská práva třetích osob.

V Praze dne 31.03.2026

Poděkování

Rád bych touto cestou poděkoval vedoucímu práce panu Ing. Martinu Havránkovi, Ph.D., za cenné a odborné rady.

Implementace GitOps pro správu cloudové infrastruktury

Abstrakt

Diplomová práce se zaměřuje na analýzu přístupů k implementaci metodologie GitOps pro správu cloudové infrastruktury. Teoretická část se věnuje vymezení pojmu GitOps, jeho principům, výhodám a porovnání s tradičními metodami správy infrastruktury. Dále jsou analyzovány nástroje, které GitOps podporují, jako například ArgoCD, Flux, Terraform či Helm. Na základě získaných poznatků je navržen modelový přístup k zavedení GitOps v cloudovém prostředí, jehož součástí je návrh architektury a procesního řízení. Práce poskytuje doporučení pro efektivní využití GitOps v praxi a přispívá k hlubšímu pochopení možností automatizace v oblasti správy moderní infrastruktury.

Klíčová slova: GitOps, cloudová infrastruktura, automatizace, Argo CD, DevOps, infrastruktura jako kód, CI/CD

Implementation of GitOps for Cloud Infrastructure Management

Abstract

The thesis focuses on analyzing approaches to the implementation of the GitOps methodology for cloud infrastructure management. The theoretical part defines the concept of GitOps, explains its core principles and advantages, and compares it with traditional infrastructure management methods. Furthermore, the thesis analyzes key tools supporting GitOps, such as ArgoCD, Flux, Terraform, and Helm. Based on the research findings, a model approach to implementing GitOps in a cloud environment is proposed, including architectural design and process flow. The work provides recommendations for the practical use of GitOps and contributes to a deeper understanding of infrastructure automation in modern IT operations.

Keywords: GitOps, cloud infrastructure, automation, Argo CD, DevOps, Infrastructure as Code, CI/CD

Obsah

1. Úvod 10
1.1. Cíl práce: 11

2 Teoretická východiska 12
2.1 Úvod do IaC – Infrastruktura jako kód 12
2.1.1 Přehled GitOps a jeho význam v cloudové infrastruktuře 12
2.2 Vývoj správy infrastruktury: manuální správa → Infrastructure as Code (IaC) → GitOps 15
2.3 Výhody GitOps: auditovatelnost, konzistence, bezpečnost 16

3. Analýza nástrojů GitOps 20
3.1 ArgoCD 20
3.2 Flux – kontinuální nasazování pomocí GitOps s Flux CD 24
3.3 Terraform – infrastruktura jako kód v GitOps procesu 27
3.4 Helm – balíčkovací systém pro Kubernetes a jeho využití v GitOps 30
3.5 Odůvodnění volby technologického stacku pro praktickou část 35

4.Implementace GitOps v cloudovém prostředí 36
4.1 Proces nasazení GitOps pro správu cloudové infrastruktury 38
4.2 Vytvoření nové aplikace v Argo CD 39
4.2.1 Integrace GitOps s GitHub a Kubernetes klastrem 41
4.3 Ověření GitOps přístupu pomocí změnového scénáře 42

5. Metodika nasazeni Gitops 44
5.1 Vyhodnocení výchozího stavu organizace z hlediska GitOps 45
5.2 Návrh technologického a procesního rámce zavedení GitOps 47
5.3 Vysvětlení principu GitOps na základě stavového modelu Gitu 50
5.3.1 Git jako stavový model a zdroj pravdy 50
5.3.2 Průběžná rekonsiliace stavu v GitOps 53
5.3.3 Vazba stavového modelu Gitu na činnost GitOps nástrojů 55
5.4 Rozšíření metodiky o ověření odolnosti a enterprise přesah (STEP 4 a STEP 5) 56
5.5 Návaznost metodiky na praktickou implementaci (kapitola 4) 56
5.5.1 Vytvořené artefakty a jejich role 56
5.5.2 Ověřené vlastnosti GitOps přístupu 57
5.5.3 Stručné zhodnocení dosaženého výsledku 57

6. Výsledky a diskuze 58
6.1 Realizace metodiky Step 1–5 v cloudovém prostředí 58
6.1.1 Step 1 – Vyhodnocení výchozího stavu 58
6.1.2 Step 2 – Založení GitOps controlleru a propojení s Git (Argo CD) 58
6.1.3 Step 3 – Git jako jediný zdroj pravdy a změnový scénář 59
6.1.4 Step 4 – Self-healing (drift → automatická náprava) 61

6.1.5 Step 5– Shrnutí ověření a doporučení pro rozšíření (governance, bezpečnost, Terraform) 63
6.2 Pozorované výsledky 64
6.3 Diskuze, limity a doporučení 65
6.4 Problémy během implementace a jejich řešení 68

7. Závěr 69

Seznam použitých zdrojů 71

Seznam obrázků, tabulek, grafů a zkratek 74

1. Úvod

V posledních letech dochází k výraznému posunu ve způsobu, jakým organizace spravují svou IT infrastrukturu. S nástupem cloud computingu a rostoucím důrazem na agilní vývoj se stále více prosazují automatizované přístupy ke správě systémů. Jedním z moderních trendů, který se v tomto kontextu ukazuje jako velmi efektivní, je metodologie GitOps.

GitOps je přístup, který využívá verzovací systém Git jako jediný zdroj pravdy pro definici a řízení infrastruktury. Změny jsou prováděny prostřednictvím pull requestů a automaticky aplikovány do produkčního prostředí pomocí nástrojů, které sledují stav repozitáře a synchronizují jej se skutečným stavem systému. Díky tomu GitOps zajišťuje vyšší úroveň transparentnosti, bezpečnosti, automatizace a auditovatelnosti oproti tradičním způsobům správy infrastruktury.

Zatímco tradiční přístupy často spoléhají na manuální konfiguraci nebo skripty bez centrálního řízení, GitOps přináší standardizovaný, deklarativní a opakovatelný model správy. Tento přístup se ukazuje jako velmi vhodný zejména v prostředí cloudových služeb a kontejnerových platforem jako Kubernetes, kde je důležitá schopnost rychlého nasazování a bezpečné obnovy stavu systému.

S rostoucí popularitou nástrojů jako ArgoCD, Flux, Terraform či Helm nabývá GitOps na významu v praxi firem různých velikostí — od start-upů po enterprise organizace. Tyto nástroje společně umožňují efektivní zavedení GitOps principů a přispívají k celkové zralosti DevOps procesů.

1.1. Cíl práce:

Úkoly výzkumu:

1. Prozkoumat koncept GitOps a jeho roli v řízení infrastruktury.
2. Porovnat GitOps s tradičními metodami správy cloudové infrastruktury.
3. Analyzovat nástroje podporující GitOps (ArgoCD, Flux, Terraform, Helm).
4. Popsat proces implementace GitOps v cloudovém prostředí.
5. Zhodnotit výhody, nevýhody a best practices GitOps v podnikovém nasazení.

Cílem této diplomové práce je detailně prozkoumat a zhodnotit možnosti využití metodologie GitOps v oblasti správy cloudové infrastruktury, zejména z pohledu automatizace, bezpečnosti, udržitelnosti a auditovatelnosti. Práce se soustředí na definici klíčových principů GitOps, jeho výhod oproti tradičním přístupům a jeho propojení s nástroji jako jsou Argo CD, Flux, Terraform či Helm.

Součástí cíle je navrhnout modelový přístup k zavedení GitOps ve firmě nebo technologickém prostředí, který bude zahrnovat nejen architektonické schéma řešení, ale také popis jednotlivých kroků, předpoklady a potenciální přínosy. Důraz je kladen na přehled současných best practices a na formulaci doporučení pro reálnou praxi.

Výsledkem práce by mělo být ucelené metodické doporučení, které může sloužit jako podklad pro firmy uvažující o zavedení GitOps přístupu v rámci svých DevOps nebo cloudových operací.

2 Teoretická východiska

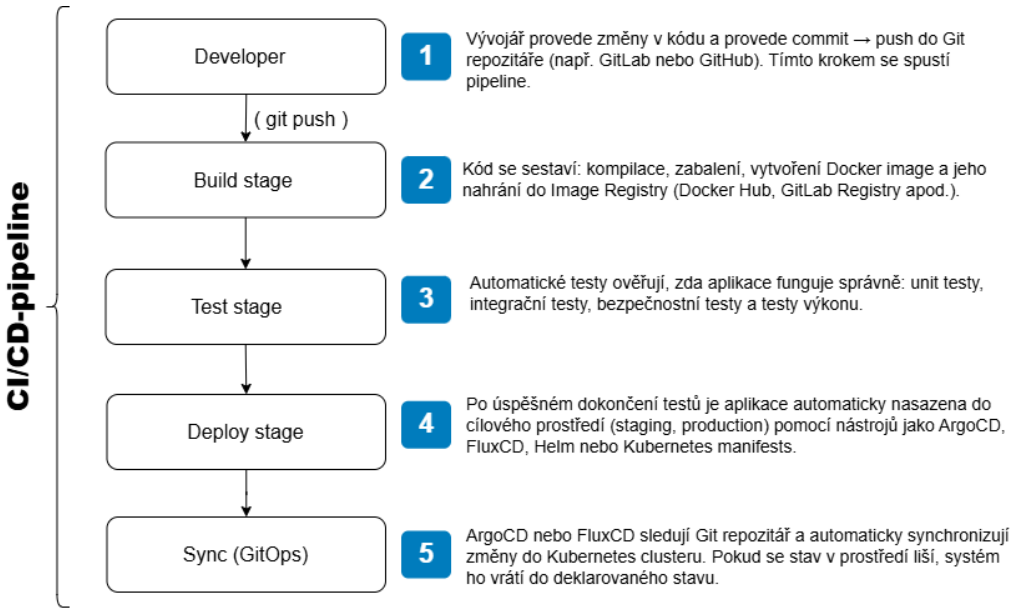
2.1 Úvod do IaC – Infrastruktura jako kód

Tato kapitola vymezuje základní pojmy a principy, na nichž stojí moderní správa cloudové infrastruktury. Nejprve stručně představíme přístup Infrastructure as Code (IaC), jehož cílem je aplikovat softwarové inženýrství na návrh, provisioning a změny infrastruktury deklarativním a verzovatelným způsobem (Kim et al., 2016). Následně vysvětlíme, jak na tyto principy navazuje GitOps – metodika, která používá Git jako jediný zdroj pravdy a automaticky slaďuje deklarovaný a skutečný stav prostředí pomocí pull-modelu a reconciliation loop (Vinto & Soto, 2023; OpenGitOps Working Group, 2021). Jako referenční platformu používáme Kubernetes, protože umožňuje deklarativní konfiguraci, vysoký stupeň automatizace a stal se de-facto průmyslovým standardem pro orchestraci kontejnerů (Arundel & Domingus, 2019). Nakonec kapitola propojuje uvedené koncepty do uceleného rámce (IaC + GitOps) a připraví půdu pro praktickou část práce, kde je tento rámec implementován na vybraném stacku.

2.1.1 Přehled GitOps a jeho význam v cloudové infrastruktuře

GitOps je metodologie, při které Git repozitáře hrají roli jediného zdroje pravdy pro správu podnikové infrastruktury jako kódu. Na základě principů DevOps poskytuje GitOps rámec pro automatizaci procesů dodávání a správy. Jednou z klíčových výhod je transparentnost – všechny změny jsou zaznamenávány v systému pro správu verzí Git, což zvyšuje reprodukovatelnost konfigurace a bezpečnost infrastruktury před nevratnými změnami nebo ztrátou dat.

Obrazek 2.1 CI/CD-pipeline



Zdroj: vlastní zpracování, Vinto & Soto (2023).

GitOps v této práci chápeme jako přístup opřený o tři základní prvky:1) Git jako jediný zdroj pravdy.Požadovaný (cílový) stav infrastruktury a aplikací je uchováván v systému správy verzí Git: patří sem manifesty Kubernetes (nejčastěji ve formátu YAML, ale mohou být také v jiných formátech, jako jsou JSON nebo HCL), politiky a omezení, šablony nasazení i popisy cloudových zdrojů. Git tak představuje „pravdu“ o tom, jak má prostředí vypadat, což zajišťuje dohledatelnost změn, reprodukovatelnost konfigurací a procesní kontrolu pomocí revizí a schvalování (Vinto & Soto, 2023, s. 1–2).2) Všechno jako kód (Everything as Code). Deklarativně a verzovaně je popisováno nejen aplikační chování, ale i infrastruktura, konfigurační artefakty, přístupové politiky, pravidla observability a i dashboardy. Tím se sjednocují inženýrské praktiky (code review, testy, lintování), zvyšuje se míra automatizace a přenositelnost řešení mezi prostředími a clustery (Vinto & Soto, 2023; OpenGitOps Working Group, 2021). 3) Operace prostřednictvím Git-workflow. Změny v prostředí probíhají standardním Git-procesem: po kontrole a sloučení pull requestu GitOps operátor (např. Argo CD či Flux) stáhne novou revizi, porovná skutečný stav clustru s tím, co je popsáno v repozitáři, a automaticky uvede systém do požadovaného stavu (reconciliation). Tento pull-přístup zvyšuje předvídatelnost a bezpečnost dodávky a zároveň snižuje riziko konfiguračního driftu (Vinto & Soto, 2023).

Použití GitOps je spojeno s nástroji, jako jsou Kubernetes, ArgoCD nebo FluxCD, které umožňují realizovat automatizovanou správu životního cyklu aplikací (OpenGitOps Working Group, 2021).

Tabulka 2.1 GitOps nástroj

Platforma	**Podpora GitOps nástrojů**	**Typ nasazení**	**Integrace certifikátů / IAM / RBAC**	**Poznámka**
AWS (Amazon Elastic Kubernetes Service)	Argo CD, FluxCD, Crossplane	GitOps + Crossplane pro infra, aplikace v EKS	Podpora AWS IAM, AWS Secrets Manager, KMS, RBAC (platformengineering.org, aws.amazon.com, CloudThat)	Pull-model, drift detection, audit v Git
Azure Kubernetes Service (AKS)	FluxCD (nativně jako AKS extension), Argo CD	GitOps operátor přímo v AKS	Integrace s Azure AD, Azure Policy, RBAC (Microsoft Learn)	Integrace s Azure AD/RBAC
Google Kubernetes Engine (GKE)	Argo CD, FluxCD	GitOps na GKE, často přes Anthos pro multi-cloud	Kompatibilita s GCP IAM, Cloud Build, Sandbox izolace (DEV Community, Medium)	Důraz na Kustomize/Helm 2–5).

Zdroj: vlastní zpracování, Vinto & Soto (2023).

2.2 Vývoj správy infrastruktury: manuální správa → Infrastructure as Code (IaC) → GitOps

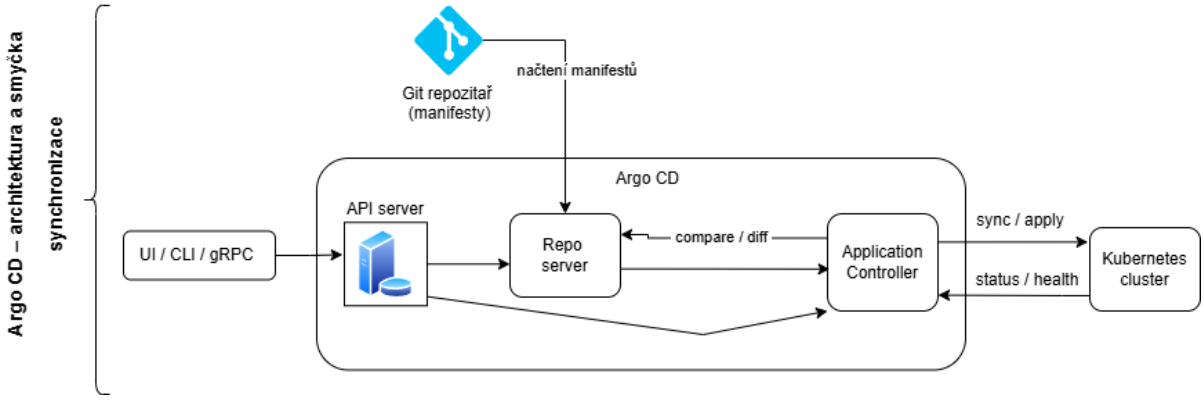
Tradiční správa IT infrastruktury se dlouho opírala o ruční postupy: administrátoři konfigurovali každý server zvlášť, což postupně vedlo k jedinečným a složitě reprodukovatelným konfiguracím a zvyšovalo riziko výpadků (Kim et al., 2016). Reakcí na tyto limity se stal přístup Infrastructure as Code (IaC): požadovaný stav se popíše deklarativně jako kód, uloží se do Gitu a automatizovaně se uplatní nástroji typu Terraform, Ansible či Puppet. Tím se omezí ruční zásahy, sníží chybovost a sjednotí konfigurace napříč prostředími (Morris, 2016; Kim et al., 2016). „Reprodukovatelnost“ zde znamená, že stejné vstupy – deklarace, pevně určené verze závislostí a deterministické buildy – vždy vedou ke stejnému výsledku; běžící instance se při změnách neopravují za provozu, ale nahrazují se předem připravenými obrazy, zatímco provozní konfigurace aplikací je zapsána v manifestech (např. Kubernetes YAML, Helm/Kustomize). Všechny změny a popisy jsou uloženy v systému kontroly verzí, zejména v Gitu (GitHub, GitLab, Bitbucket, Azure Repos, AWS CodeCommit, Gitea), který v GitOps funguje jako jediný zdroj pravdy a auditní kniha (Vinto & Soto, 2023; Kim et al., 2016).

Na IaC navazuje GitOps. Změny se zaznamenají jako commity (uložené sady úprav s autorem, časem a zprávou) a slučují se přes pull request, který nejprve prochází code review – kontrolou určenými recenzenty. Ti vyhodnotí rozdíly (diff) mezi verzemi, výsledky automatických kontrol (buildy, testy, politiky) a soulad s interními pravidly; teprve pak se změna slije do cílové větve. Běžící systémy jsou poté automaticky uváděny do souladu s deklarovaným stavem v repozitáři prostřednictvím „smyčky porovnání a nápravy“ (reconciliation loop) – řídicí komponenta pravidelně porovná reálný a požadovaný stav a při zjištěném rozdílu provede nápravu. V prostředí Kubernetes tuto mechaniku realizují zejména Argo CD a Flux (Vinto & Soto, 2023; OpenGitOps WG, 2021).

Podporu GitOps dále rozšiřují poskytovatelé cloudu: v AKS je Flux dostupný jako rozšíření, GKE nabízí Config Sync pro centrální správu flotil, a v EKS jsou běžné integrace pro bootstrap a životní cyklus clusterů pomocí Crossplane ve spojení s Argo CD/Flux. Tím se umožní jednotná správa více clusterů a prostředí, detekce driftu a plně auditovatelná historie změn (Microsoft Learn, 2025; Google Cloud Docs, 2025; AWS Containers Blog, 2021–2023). Do GitOps cyklu lze také vložit policy-as-code (např. OPA Gatekeeper) a vynucovat organizační pravidla už na úrovni admission control, čímž se posiluje bezpečnost a dodržování politik napříč flotilou (Kubernetes Blog/OPA Gatekeeper, 2019). Výsledkem je přesun od pracné a chybové ruční správy k reprodukovatelným, auditovatelným a škálovatelným procesům řízeným kódem a

možností využívat systémy kontroly verzí (Kim et al., 2016; Vinto & Soto, 2023)

Obrázek 2.2.: Argo CD – architektura a smyčka synchronizace

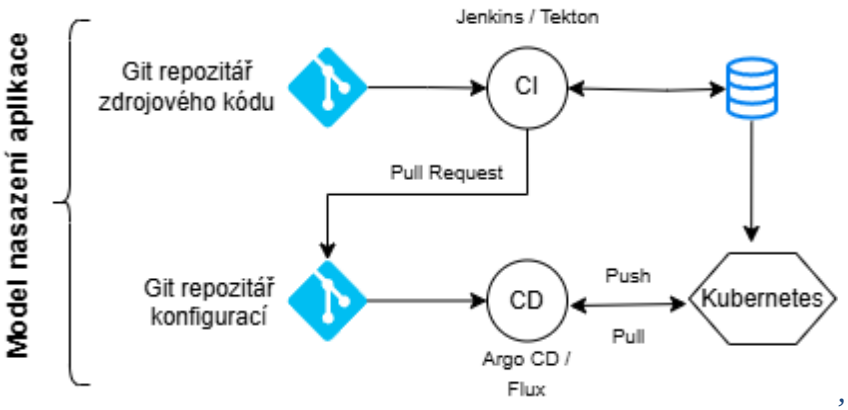


Zdroj: vlastní zpracování podle dokumentace Argo CD.

2.3 Výhody GitOps: auditovatelnost, konzistence, bezpečnost

GitOps přináší řadu výhod, které se projevují především v oblasti auditovatelnosti, konzistence konfigurací a bezpečnosti provozu.V prostředí, kde jsou všechny změny infrastruktury spravovány přes Git, se každá úprava zapisuje do verzovacího systému spolu s autorem, časem i popisem změny. Tento mechanismus vytváří kompletní historii infrastruktury a umožňuje zpětně dohledat původ jakéhokoli incidentu. V praxi to znamená, že infrastruktura je plně auditovatelná, změny jsou přehledně dokumentovány a v případě potřeby je možné systém obnovit do předchozího funkčního stavu. Transparentnost procesů výrazně snižuje riziko chyb způsobených manuální konfigurací a zajišťuje, že infrastruktura odpovídá deklarovaným požadavkům (Vinto & Soto, 2023; OpenGitOps Working Group, 2021).

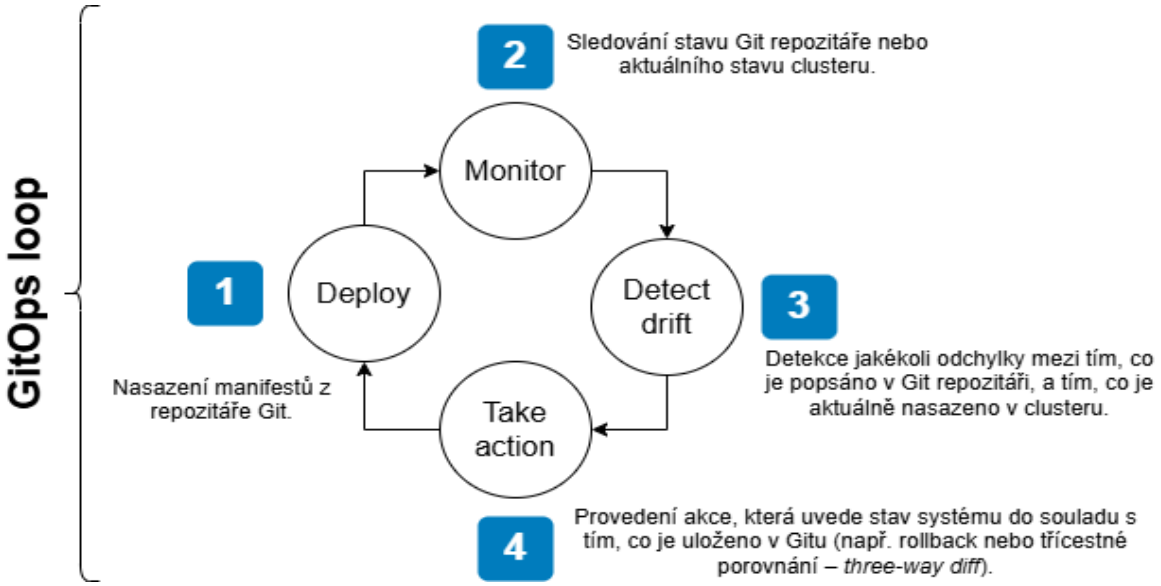
Obrázek 2.3. Model nasazení aplikace (push vs. pull)



Zdroj: vlastní zpracování, Vinto & Soto (2023).

Další klíčovou vlastností GitOps je zajištění konzistence mezi deklarovaným a skutečným stavem. GitOps operátor (např. Argo CD nebo Flux) v reálném čase porovnává konfiguraci prostředí uloženou v repozitáři s běžícím clusterem. Pokud zjistí odchylku – tzv. *drift* – automaticky ji opraví a obnoví žádoucí stav. Tento mechanismus, označovaný jako *reconciliation loop*, činí správu prostředí předvídatelnou a reprodukovatelnou napříč všemi fázemi životního cyklu aplikace (dev, test, prod). GitOps tak eliminuje problematiku ručně udržované serveru a zajišťuje jednotné chování mezi clustery. Praktická implementace v nástrojích Argo CD a Flux CD zahrnuje detekci OutOfSync, automatickou synchronizaci a detailní záznam o provedených akcích (Vinto & Soto, 2023; Argo CD Docs; Flux Docs).

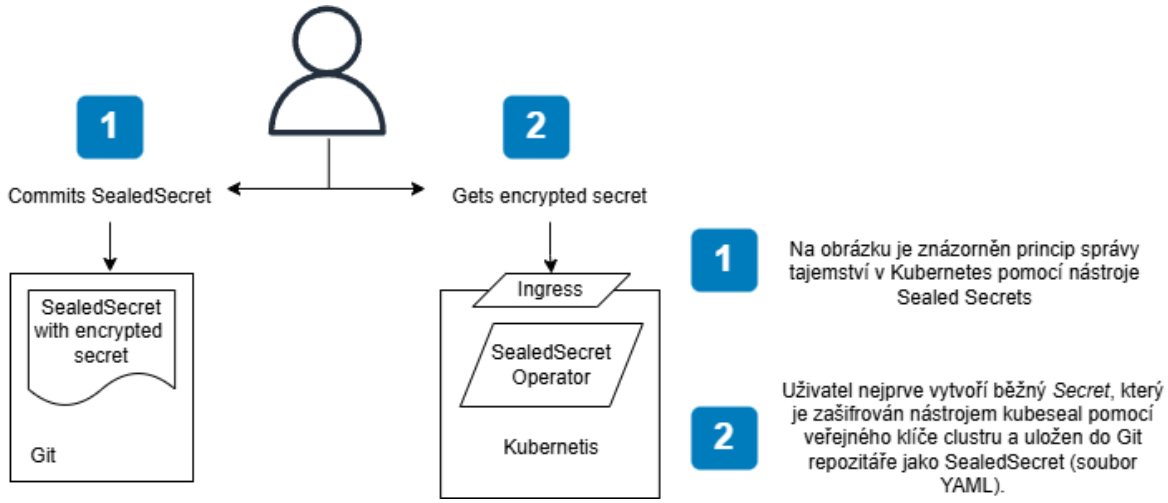
Obrazek 2.4 GitOps loop



Zdroj: vlastní zpracování, Vinto & Soto (2023).

V oblasti bezpečnosti GitOps podporuje přístup secrets as code “a „policy as code “. Citlivé údaje (např. API klíče, certifikáty) se do repozitáře ukládají v šifrované podobě pomocí nástrojů jako Sealed Secrets, kde se dešifrování provádí až v cílovém clusteru. Tím je zajištěno, že tajemství nejsou nikdy vystavena v otevřené formě. Alternativně lze využít External Secrets Operator, který propojuje Kubernetes s externími trezory (HashiCorp Vault, AWS Secrets Manager apod.) a předává hodnoty dynamicky během nasazení. Paralelně lze prosazovat organizační politiky prostřednictvím nástroje OPA Gatekeeper, jenž deklaračně vymezuje pravidla pro nasazované zdroje, monitoruje jejich soulad se standardy a zajišťuje auditní záznamy (policy compliance). Tímto způsobem GitOps spojuje vysokou úroveň automatizace s důrazem na bezpečnost a dodržování pravideyl (Bitnami Sealed Secrets; Red Hat Docs; OPA Gatekeeper).

Obrazek 2.5 Správa tajemství pomocí Sealed Secrets v GitOps



Zdroj: vlastní zpracování, Vinto & Soto (2023).

Je třeba poznamenat i potenciální obtíže při zavádění GitOps. Ve velkorozsáhlých systémech může vzrůst administrativní zátěž: počet repozitářů a CI/CD-pipeline roste, což vyžaduje další nástroje pro orchestraci změn a řízení přístupu (Sándor, 2020). Kromě toho GitOps automaticky neřeší otázku správy secretů – důvěrná data (hesla, klíče) se obvykle ukládají mimo Git s využitím speciálních úložišť, což přidává vrstvu integrace (Sándor, 2020). Nakonec přechod k modelu GitOps předpokládá vyspělost procesů: organizace musí být připravena na celkové řízení verzí a code review všech změn. Bez kultury DevOps a zavedených praktik samo o sobě použití GitOps neodstraní stávající problémy a jeho přínosy nebudou realizovány v plné míře.

3. Analýza nástrojů GitOps

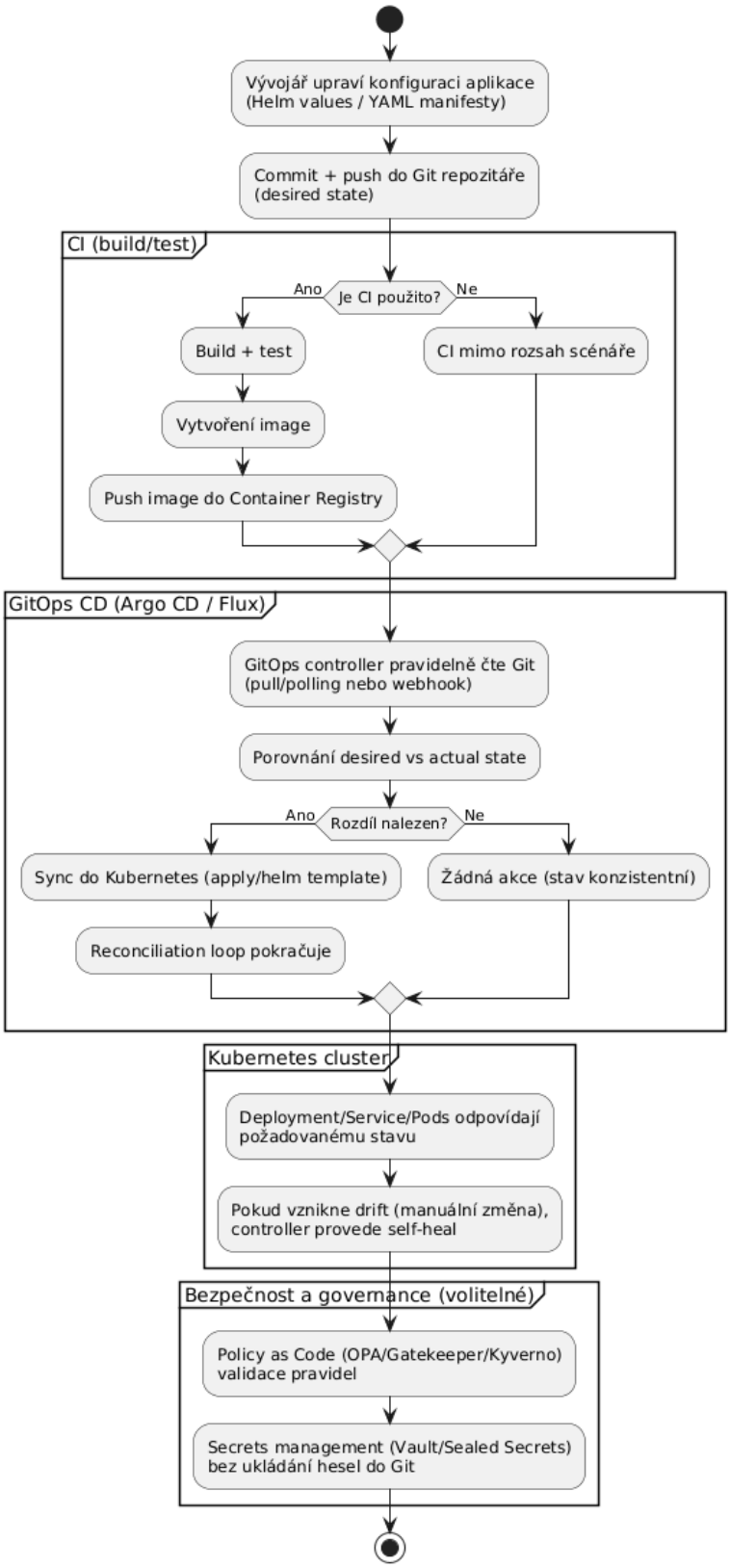
V této kapitole je provedena analýza nástrojů spojených s metodologií GitOps. Ačkoliv ekosystém cloud-native technologií nabízí širokou škálu alternativních řešení (např. Pulumi či Crossplane pro infrastrukturu, nebo Kustomize pro správu manifestů), následující části podrobně popisují čtyři vybrané nástroje – Argo CD, Flux, Terraform a Helm.

Tato čtveřice byla zvolena proto, že v současnosti představuje de-facto průmyslový standard pro implementaci GitOps: Terraform zajišťuje spolehlivou přípravu základní cloudové infrastruktury, Helm poskytuje standardizovaný způsob balíčkování aplikací a Argo CD s Fluxem reprezentují dva nejrozšířenější a nejvyzrálejší nástroje (podpořené nadací CNCF) pro samotnou pull-based synchronizaci stavu v prostředí Kubernetes. Jejich analýza poskytuje ucelený pohled na to, jak lze plně automatizovat životní cyklus od infrastruktury až po aplikační vrstvu.

3.1 ArgoCD

Argo CD je deklarativní nástroj pro kontinuální doručování (Continuous Delivery, CD) postavený pro Kubernetes, který implementuje principy GitOps (Vinto & Soto, 2023). Argo CD běží jako kontrolér v Kubernetes clusteru a průběžně sleduje změny v definicích aplikací uložených v Git repozitáři (Arundel, Domingus, 2019). Pokud zjistí odchylku mezi verzovaným požadovaným stavem v Git a aktuálním stavem clusteru, provede automatickou synchronizaci, aby cluster uvedl do požadovaného stavu (Forsgren et al., 2018). Tím eliminuje potřebu manuálních nasazení a snižuje riziko konfiguračních odchylek (Kim et al., 2016).

Obrázek 3.1 – Schematické znázornění GitOps workflow



Zdroj: vlastní zpracování

V GitOps procesu vývojář provede změnu kódu nebo konfigurace a odešle ji do Git repozitáře (krok 1). GitOps operátor (například Argo CD nebo Flux) pravidelně pull-uje deklarovaný stav z Git repozitáře (krok 2) a porovnává jej s běžícím prostředím. Nalezené změny nebo odchylky následně aplikuje do cílového prostředí (krok 3), zde Kubernetes clusteru. Operátor zároveň monitoruje aktuální stav clusteru a vyhodnocuje jeho soulad s požadovaným stavem (průběžné monitorování v rámci smyčky zpětné vazby). Tento postup zajišťuje, že produkční prostředí je kontinuálně sladěno s obsahem Git repozitáře a jakákoli odchylka (tzv. drift) je včas odhalena a odstraněna (Vinto & Soto, 2023; Forsgren et al., 2018). GitOps přístup tak přináší standardizovaný workflow pro nasazování a rollbacky, vyšší bezpečnost díky povinným code-review při změnách v Git a auditovatelnost všech změn (Davis, Daniels, 2016).

Argo CD se vyznačuje „pull" modelem nasazování – na rozdíl od tradičních push CI/CD pipeline zde samotný cluster prostřednictvím Argo CD aktivně stahuje a aplikuje změny z Git repozitáře (Yuen et al., 2021, s. 245). Tím je dosaženo vyšší robustnessi: pokud dojde k výpadku CI serveru nebo krátkodobě k nedostupnosti clusteru, systém dožene nasazení změn při nejbližší synchronizaci jakmile jsou podmínky obnoveny (Vinto a Soto, 2023). Argo CD podporuje několik mechanismů pro definici manifestů – kromě prostých YAML manifestů Kubernetes umí pracovat i s Kustomize, Helm šablonami či Jsonnet, což umožňuje snadno integrovat do GitOps procesu i komplexnější konfigurace (Yuen et al., 2021; Vinto & Soto, 2023).

Hlavní vlastnosti Argo CD: (podle Vinto & Soto, 2023)

- Deklarativní cílový stav: Uživatelé definují požadovaný stav aplikací a infrastruktury deklarativně (typicky jako YAML manifesty) verzované v Git. Argo CD tyto definice považuje za zdroj pravdy a všechny změny v prostředí jsou odvozeny z Git commitů, nikoli z ručních zásahů.
- Kontinuální synchronizace: Argo CD periodicky (či na základě webhooku) kontroluje Git repozitář a zjišťuje, zda deployed prostředí odpovídá deklarovanému stavu. V případě rozdílu označí aplikaci jako *OutOfSync* a může automaticky provést synchronizaci (nasazení změn) nebo vyčkat na ruční potvrzení, dle konfigurace politik (Vinto & Soto, 2023).
- Stav a health management: Pro každou spravovanou aplikaci sleduje Argo CD nejen synchronizační stav (In Sync / Out of Sync), ale i zdravotní stav (Healthy / Degraded apod.)

jednotlivých Kubernetes objektů. Pomocí vestavěných zdravotních checks (např. čekání na Healthy stav Podů, Deploymentů atd.) umí vyhodnotit, zda nasazení proběhlo úspěšně (Yuen et al., 2021).

- Webové UI a auditovatelnost: Argo CD poskytuje webové uživatelské rozhraní, ve kterém lze přehledně vizualizovat stromy závislostí aplikací, jejich stav, rozdíly oproti Git apod. (Arundel, Domingus, 2019). Díky verzování v Git a historii synchronizačních akcí v Argo CD je každá změna auditovatelná – je dohledatelné, která verze kódu byla nasazena, kdy a kým schválena (Forsgren et al., 2018).
- Bezpečnost a přístupová práva: Argo CD podporuje integraci s podnikových SSO (Single Sign-On) systémy a umožňuje definovat RBAC pravidla pro přístup uživatelů (např. práva spouštět synchronizaci, upravovat projekty, atd.) (Yuen et al., 2021). Declarative setup umožňuje i konfiguraci samotného Argo CD provádět přes Git – tzv. *App of Apps* pattern, kdy seznam aplikací spravovaných Argo CD je také verzován v Git repozitáři a Argo CD se při startu z něj inicializuje (Vinto & Soto, 2023). Tím lze Argo CD snadno bootstrapovat na novém clusteru a udržovat jeho konfiguraci konzistentní mezi prostředími.

Argo CD je široce využíván v cloud-native prostředí a stal se jedním z nejpopulárnějších nástrojů pro GitOps nasazování díky své spolehlivosti a bohaté sadě funkcí (Kim et al., 2016). Podle autorů knihy *GitOps Cookbook* přináší Argo CD organizacím schopnost zvýšit frekvenci nasazování až na stovky denně a současně zlepšit stabilitu – všechny změny procházejí Git workflow, což zvyšuje důvěru v nasazovací proces a snižuje MTTR (Mean Time to Recovery) při incidentech (Vinto & Soto, 2023). Tyto výhody odpovídají závěrům studie *Accelerate*, která identifikuje časté a spolehlivé nasazování jako znak elitně výkonných IT týmů (Forsgren et al., 2018). Implementace nástroje jako Argo CD v souladu s DevOps principy (průběžná integrace, průběžné doručování, infrastruktura jako kód) umožňuje těchto výkonnostních metrik dosáhnout (Forsgren et al., 2018; Kim et al., 2016).

Z hlediska architektury se Argo CD skládá z několika komponent běžících v clusteru: API server (gRPC/REST server poskytující API pro Web UI, CLI a integraci s CI), Application Controller (řídí synchronizaci stavu aplikací, porovnává Git vs. cluster a provádí potřebné akce) a Repository Server (zajišťuje přístup k Git repozitářům, načítání manifestů, jejich templating pomocí Helm/Kustomize atd.) (Yuen et al., 2021; Arundel, Domingus, 2019). Díky této architektuře může Argo CD škálovat na správu velkého počtu aplikací i více Kubernetes clusterů zároveň – jeden Argo CD instance je schopna deployovat aplikace i do vzdálených clusterů, pokud má potřebná přístupová oprávnění (Vinto & Soto, 2023). To usnadňuje multi-

cluster scénáře, kde centrální Argo CD synchronizuje stavy do desítek či stovek clusterů pro různé týmy či prostředí při zachování konzistence (Yuen et al., 2021).

Na závěr lze říci, že Argo CD výrazně zjednodušuje proces nasazování cloud-native aplikací. Umožňuje deklarativní a plně automatizované nasazení, kde jak vývojové, tak operační týmy pracují se sdíleným, verzovaným popisem infrastruktury a aplikací. Tento přístup minimalizuje riziko lidské chyby, zvyšuje rychlost dodávek a zlepšuje transparentnost – každý krok je viditelný v Git historii (Kim et al., 2016). Argo CD tak naplňuje filosofii DevOps v praxi: propojuje vývoj s provozem pomocí společných nástrojů a procesů a podporuje *nepřetržitou spolupráci* nad změnami v kódu i infrastruktuře (Davis, Daniels, 2016).

3.2 Flux – kontinuální nasazování pomocí GitOps s Flux CD

Flux (někdy označovaný jako Flux CD) představuje další open-source nástroj k realizaci GitOps přístupu pro správu aplikací v Kubernetes. Vytvořila jej společnost Weaveworks, která stála i u zrodu samotného pojmu GitOps (Vinto & Soto, 2023). Flux pracuje na podobném principu jako Argo CD – v pravidelných intervalech (nebo na základě webhook signálů) vyhledává změny v definicích požadovaného stavu uložených v Git repozitáři a provádí jejich aplikaci do Kubernetes clusteru (Yuen et al., 2021). Flux tak zajišťuje, že deklarativně popsané konfigurace aplikací jsou neustále prosazovány v prostředí, čímž eliminuje *drift* a udržuje synchronizaci mezi Git a produkcí (Forsgren et al., 2018).

Na rozdíl od Argo CD, který nabízí ucelenou aplikaci s webovým GUI, Flux byl navržen spíše jako lightweight toolkit složený z několika specializovaných kontrolérů v Kubernetes (Arundel, Domingus, 2019). Ve své nejnovější verzi (Flux v2) je modulární – jádro tvoří Source Controller (zajišťuje přístup ke zdrojům deklarací, jako je Git repozitář nebo Helm repository), Kustomize Controller a Helm Controller (aplikují manifesty do clusteru buď přímo nebo renderováním Helm chartů), dále Notification Controller (pro webhooky a oznámení) a volitelně Image Automation Controllers (pro práci s kontejnerovými obrazy) (Yuen et al., 2021). Díky modulární architektuře lze Flux nasadit jen s potřebnými komponentami a dobře jej škálovat i přizpůsobovat potřebám (Vinto & Soto, 2023).

Flux nepoužívá vlastní webové rozhraní – správa probíhá prostřednictvím deklarativních zdrojů (CRD) v Kubernetes a pomocí příkazové řádky flux CLI nebo integrací do jiných nástrojů (Yuen et al., 2021). To znamená, že konfigurace toho, co a odkud se má nasazovat, je opět deklarativní a verzovaná: uživatel vytvoří v clusteru objekty typu GitRepository, Kustomization či HelmRelease, které určují, z jakého repozitáře a jaké cesty se mají manifesty brát a jak je aplikovat (Arundel, Domingus, 2019). Flux kontrolér pak tyto objekty čte a provádí podle nich akce – například GitRepository specifikuje URL repozitáře a

revizi (commit/tag), Kustomization odkazuje na GitRepository a cestu k manifestům (případně Kustomize patchům) a určuje cíl aplikace v clusteru (namespace atd.). Toto plně deklarativní řízení nasazení skrze Kubernetes CRD objekty umožňuje tzv. GitOps pro GitOps – samotná konfigurace CD pipeline je verzovaná v Git stejně jako aplikace (Yuen et al., 2021).

Mezi výhody Fluxu patří jeho úzká integrace s ekosystémem Kubernetes – využívá Kubernetes API a objekty k reprezentaci konfigurace, což zapadá do standardních postupů správy clusteru (Vinto & Soto, 2023). Flux je také velmi nenáročný a vhodný pro embedded scénáře nebo edge nasazení, kde plnohodnotné UI není potřeba. Ve spojení s nástrojem Flux CLI nebo s doplňkovými UI (např. Weave GitOps) však uživatelé mohou získat přehled o stavu podobně jako u Argo CD. Další silnou stránkou Fluxu je zabudovaná podpora pro automatizaci aktualizace verzí obrazů (tzv. Image Update Automation) (Yuen et al., 2021). Flux dokáže monitorovat kontejnerové registry a při zjištění nové verze obrazu (např. na základě tagu) umí automaticky provést aktualizaci příslušného manifestu v Git repozitáři (úpravou image tagu v YAML souboru) a commitnout tuto změnu (Vinto & Soto, 2023). Tím lze dosáhnout plně automatizovaného continuous deploymentu, kde nový build aplikace spustí aktualizaci manifestů a jejich aplikaci do clusteru bez nutnosti ručního zásahu. Tato funkčnost není v Argo CD nativně přítomna (tam by bylo třeba kombinovat s jiným nástrojem, nebo použít Argo Rollouts pro progressive delivery), což je jedna z významných odlišností mezi Flux a Argo CD.

Flux i Argo CD sdílejí deklarativní GitOps filozofii – oba přístupy vedou k vyšší stabilitě a auditovatelnosti nasazení. Rozdíly lze shrnout tak, že Argo CD poskytuje více uživatelsky komfortní all-in-one řešení (vhodné např. pokud tým preferuje vizuální rozhraní a snadnou správu více aplikací skrze UI), zatímco Flux cílí na flexibilitu a kubernetes-native způsob – konfigurace jako kód, minimum externích závislostí a možnost rozšíření (Yuen et al., 2021). Podle Accelerate metrik DevOps zralosti nejsou nástroje jako Argo či Flux samy o sobě cílem, ale prostředkem k dosažení častých nasazení a rychlé obnovy po chybě (Forsgren et al., 2018). Výběr konkrétního nástroje závisí na kontextu – některé organizace upřednostní Flux pro jeho jednoduchost a modulárnost, jiné Argo CD pro bohaté funkce a komunitní podporu. Důležité je, že obě řešení eliminují ruční konfigurační kroky a umožňují tzv. self-healing infrastrukturu: pokud někdo provede neautorizovanou změnu přímo v clusteru, GitOps operátor ji detekuje a automaticky vrátí do stavu definovaného v Git (Vinto & Soto, 2023). Tím je zajištěna vysoká míra kontroly a konzistence prostředí.

Flux je, podobně jako Argo CD, projekt hostovaný v CNCF (Cloud Native Computing Foundation) a dosáhl stádia Graduated, což potvrzuje jeho zralost a široké nasazení v praxi. Je používán např. ve spojení s platformou Weaveworks GitOps nebo integračně v Flux subsystem

for Argo (Flamingo), který kombinuje silné stránky obou nástrojů (Flux pro Terraform a infra, Argo pro app deploy) (Yuen et al., 2021). To reflektuje trend, že GitOps není omezeno jen na správu Kubernetes manifestů – Flux rozšiřuje působnost i na další oblasti jako je automatizace infrastruktury (viz Terraform Controller ve Flux v2, o němž bude zmínka dále v textu).

Z hlediska praktického použití vyžaduje nasazení Fluxu určitou počáteční konfiguraci (manifesty pro CRD a Controlleři). Jakmile je však Flux nasazen v clusteru, veškeré další operace – přidání nové aplikace, změna zdroje – se provádějí opět Git commitem (Arundel, Domingus, 2019). Například chceme-li nasadit novou aplikaci pomocí Flux, vytvoříme Git repozitář s její konfigurací a poté v infrastructure repozitáři (centrálním GitOps repu) přidáme manifest GitRepository a Kustomization/HelmRelease odkazující na tento nový repozitář. Flux detekuje nový GitRepository objekt, stáhne obsah repozitáře a Kustomize Controller provede aplikaci manifestů. Celý proces je tudíž plně řízen změnami v Gitu, což je stěžejní princip GitOps (Davis, Daniels, 2016).

V souhrnu, Flux poskytuje lehký, skriptovatelný nástroj pro GitOps, který je vhodný pro týmy preferující plnou infrastrukturu jako kód a vysokou míru automatizace. Stejně jako u Argo CD se jeho použitím výrazně zvyšuje frekvence nasazování a zkracuje lead time pro změny: nasazení nové verze aplikace se redukuje na merge požadované změny do Git repozitáře, o vše ostatní se postará automatizace (Forsgren et al., 2018). Flux tak pomáhá naplňovat DevOps zásady *Continuous Delivery* a *Infrastructure as Code* v praxi a přispívá ke kultuře, kde jsou změny malé, časté a reverzibilní (Kim et al., 2016). Důležitá je však i změna procesů a mindsetu v týmu – nástroje jako Flux samy o sobě nezaručí zlepšení, pokud nejsou doprovázeny vhodně nastavenými postupy (code review, testování, monitoring) a podporou vedení pro DevOps filozofii (Davis, Daniels, 2016). V kombinaci s těmito kulturními změnami představuje Flux mocný nástroj pro zvýšení efektivity a spolehlivosti softwarového dodavatelského procesu.

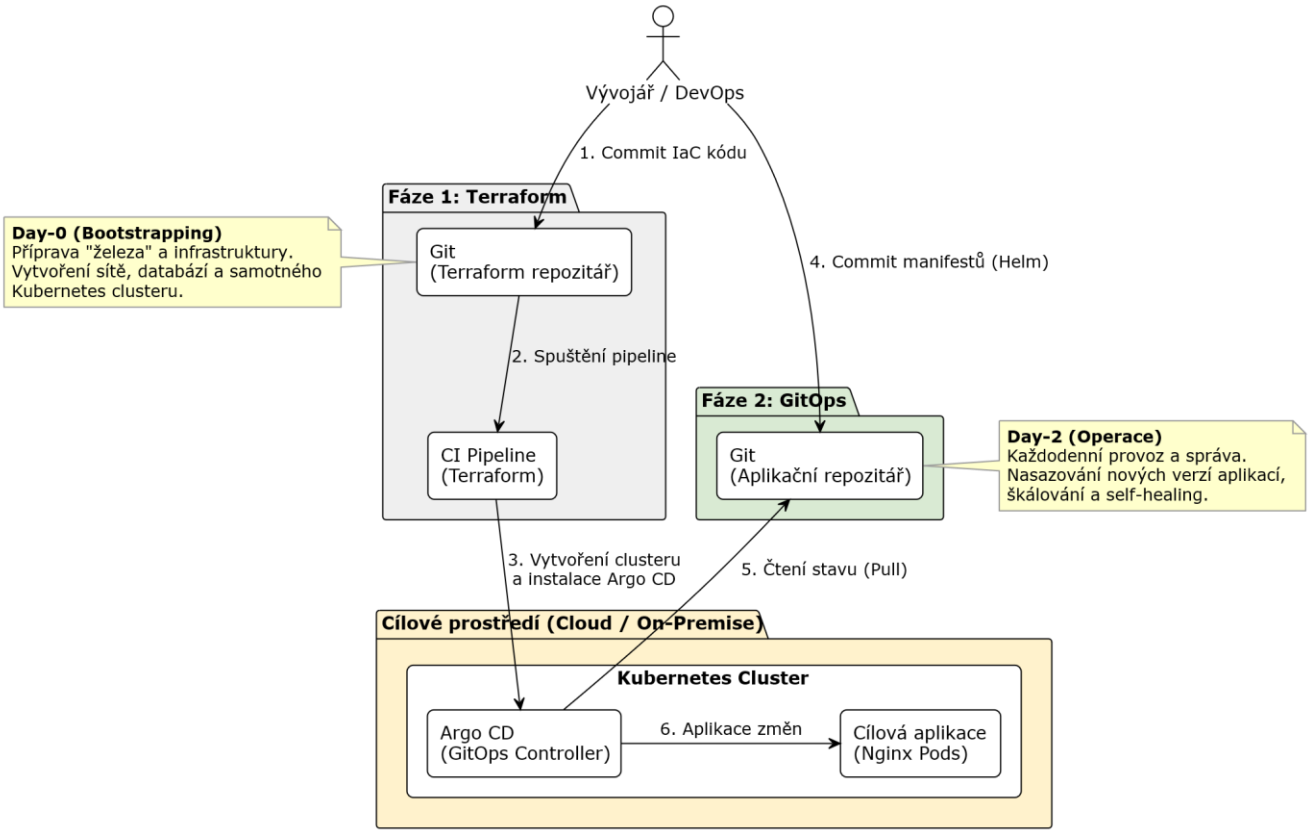
3.3 Terraform – infrastruktura jako kód v GitOps procesu

Terraform od společnosti HashiCorp je nástroj pro Infrastructure as Code (IaC), který umožňuje deklarativně popisovat a provisionovat infrastrukturu napříč různými cloudovými platformami (Hafner, 2025). Pomocí Terraformu lze spravovat zdroje jako jsou virtuální servery, kontejnery, síťové prvky či databáze, a to jednotným způsobem pomocí konfiguračních souborů ve formátu HCL (HashiCorp Configuration Language). Klíčovou vlastností je, že celý popis infrastruktury je verzovatelný a infrastrukturní změny prochází podobným workflow jako vývoj aplikací.

Terraform funguje na principu porovnání aktuálního a požadovaného stavu: načte aktuální stav infrastruktury a porovná jej s deklarovanou konfigurací v Gitu. Následně vygeneruje plán změn (terraform plan), který explicitně vypíše, jaké zdroje budou vytvořeny či zrušeny. Po schválení plánu provede aplikaci změn (terraform apply) automaticky voláním příslušných API jednotlivých poskytovatelů (Vinto & Soto, 2023).

V kontextu GitOps se Terraform uplatňuje zejména v počáteční fázi. Slouží k vytvoření samotné základní infrastruktury, na níž pak nástroje jako Argo CD nasazují aplikační vrstvu.

Obrázek 3.2 - Integrace Terraform a GitOps (Argo CD/Flux) v nasazovacím procesu



Zdroj: vlastní zpracování

Diagram znázorňuje dvoufázový pipeline: v *první fázi* se spravuje infrastruktura jako kód pomocí Terraformu – vývojář provede změnu v Terraform repozitáři (1), CI pipeline vygeneruje a aplikuje Terraform plán (2), čímž vytvoří nebo upraví infrastrukturu v cloudu (např. zřídí nový Kubernetes cluster, nastaví sítě, databáze) (3). Součástí tohoto kroku může být i automatická instalace GitOps nástroje (Argo CD/Flux) do nově vzniklého clusteru. Ve *druhé fázi* přichází na řadu aplikační GitOps: vývojář upraví konfigurace aplikace v Git repozitáři (4), GitOps operátor (Argo CD/Flux), který již běží v clusteru, detekuje změnu (5) a provede nasazení aplikace či konfigurace do Kubernetes clusteru (6). Terraform tedy řeší bootstrapping prostředí a infrastruktury, zatímco Argo CD/Flux následně zajišťují aplikování aplikačních manifestů a jejich udržování v synchronizaci.

Terraform samotný není GitOps nástroj v užším smyslu – na rozdíl od Argo CD nebo Flux nepracuje nepřetržitě jako kontrolér, ale typicky je spouštěn na vyžádání (například skrze CI pipeline při pushi do repozitáře) (Spacelift, 2023). Přesto se principy GitOps dají aplikovat i na Terraform: verze Terraform konfigurace v Gitu je považována za zdroj pravdy infrastruktury a jakýkoli změnový požadavek (pull request) v infra repozitáři se stává jediným způsobem, jak infrastrukturu měnit. Některé nástroje dokonce integrují Terraform s GitOps

kontinuálním během – například Flux Terraform Controller umožňuje provozovat Terraform plány přímo jako Kubernetes Custom Resource v clusteru (Flux, 2024). To znamená, že změna Terraform konfigurace v Git může být detekována Fluxem, který pak spustí Terraform apply a výsledek opět uloží jako Kubernetes manifesty. Takové schéma (zatím experimentální) stírá hranici mezi infrastrukturou a aplikacemi v GitOps modelu.

V podnikové praxi je však běžnější oddělovat běh Terraformu od aplikačních GitOps operátorů. Terraform je typicky spouštěn asynchronně v rámci CI/CD pipeline nebo prostřednictvím specializovaných orchestrátorů, jako je open-source nástroj Atlantis (Hafner, 2025). Po zprovoznění základní infrastruktury a clusterů je správa plynule předána nástrojům typu Argo CD nebo Flux. Důvodem této separace je odlišná povaha infrastrukturních změn: operace jako zřízení databáze či virtuálního stroje jsou časově náročnější, vyžadují transakční přístup ke stavovému souboru (state locking) a často i manuální schválení vygenerovaného plánu (Hafner, 2025). Tento proces fundamentálně kontrastuje s chováním GitOps nástrojů, které operují na bázi nepřetržité rekonsiliační smyčky (continuous reconciliation) a aplikují změny téměř okamžitě (Vinto a Soto, 2023).

Příklad konfigurace v jazyce HCL (Terraform):

```
# Definice virtuálního serveru v AWS pomocí Terraform
provider "aws" {
  region = "eu-central-1"
}
resource "aws_instance" "web_server" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t2.micro"
  tags = {
    Name = "WebServer01"
    Environment = "Dev"
  }
}
```

Listing 3.1: Ukázka Terraform konfigurace – poskytovatel AWS a jedno EC2 instanční prostředí.

Tento HCL kód deklaruje poskytovatele AWS (v regionu EU-Central) a vytváří jednu instanci virtuálního serveru s definovaným AMI ID a typem. Takovýto popis je verzován v Git a při spuštění Terraformu se vytvoří server dle parametrů. Pokud by byl server již vytvořen a někdo ručně změnil jeho parametry, příští běh terraform plan by detekoval odchylky (tzv. state drift) od tohoto deklarovaného stavu a nabídl jejich automatickou opravu (Hafner, 2025; Vinto a Soto, 2023).

V GitOps pipeline je sekvence Terraform -> Argo CD / Flux poměrně častá: Terraform se postará o tzv. Day-0 operace (příprava infrastruktury), zatímco GitOps operátory se starají o

Day-1 a dále (nasazení aplikací, konfigurací a jejich průběžné udržování). Toto oddělení odpovídá i rozdělení domén znalostí: často infrastrukturní tým připraví platformu a aplikační týmy pak v modelu self-service nasazují své aplikace přes GitOps do jim vyhrazených prostorů (Forsgren et al., 2018). Důležité je, že obě části jsou pokryty IaC principy a verzovací historií, což výrazně zvyšuje důvěryhodnost prostředí – celý stack od sítě až po aplikaci je definován kódem a je kdykoli replikovatelný (Kim et al., 2016).

Terraform podporuje kromě hlavních cloudových platforem také definování Kubernetes zdrojů. To by teoreticky umožnilo spravovat aplikace Terraformem místo GitOps operátoru. Praxe však ukázala, že kombinace obou nástrojů efektivně využívá specifických předností obou řešení: Terraform bezpečněji spravuje komplexní závislosti a orchestraci mimo cluster, zatímco Argo CD a Flux lépe obsluhují vnitřní konzistenci clusteru a reagují rychleji na změny provedené vývojáři (Hafner, 2025; Yuen et al., 2021). Moderní DevOps prostředí proto často využívá Terraform a GitOps nástroje komplementárně – Terraform vytvoří základní infrastrukturu a GitOps operátory následně obsluhují aplikační vrstvu.

3.4 Helm – balíčkovací systém pro Kubernetes a jeho využití v GitOps

Helm je nejrozšířenější nástroj pro správu balíčků aplikací v Kubernetes – často bývá označován jako *package manager* pro Kubernetes (Arundel, Domingus, 2019). Umožňuje zabalit kolekci Kubernetes manifestů (Deploymenty, Servicy, ConfigMapy apod.) do jednoho přenositelného balíčku zvaného Chart, který může být verzován a opakovaně nasazován s různými konfiguracemi. Helm výrazně usnadňuje nasazování složitějších aplikací, které obsahují více vzájemně provázaných komponent, a dovoluje uživatelům přizpůsobit parametry nasazení pomocí tzv. Values (Hodnot) bez nutnosti zasahovat do samotných manifestů (Vinto & Soto, 2023).

Každý Helm chart je v podstatě šablona (sada šablon) Kubernetes manifestů napsaných v templating jazyce (Go templates) a doplněná o implicitní proměnné a strukturu adresářů. Uživatel připraví soubor values.yaml obsahující konkrétní hodnoty parametrů (např. počet replik, obraz kontejneru, povolené moduly apod.), a Helm z těchto hodnot vygeneruje finální manifesty, které pak aplikuje do clusteru (Arundel, Domingus, 2019). Tím se výrazně redukuje duplikace kódu: pro více nasazení stejné aplikace (třeba do různých prostředí) se využije stejný chart a liší se jen values soubory. Helm také řeší správu verzí – každý chart má svou verzi a při instalaci do clusteru vytváří entitu zvanou Release, což je konkrétní instance nasazeného chartu s určitými values (Kim et al., 2016). Helm umí releasy upgradovat (aplikovat novou verzi chartu nebo jen změnu values), případně revertovat (helm rollback) v případě

problémů.

V GitOps kontextu se Helm využívá dvěma hlavními způsoby:

1. **Helm jako součást CI pipeline:** Tým může udržovat Helm charty aplikací v repozitáři a CI pipeline může při nové verzi aplikace vygenerovat manifesty pomocí helm template a ty commitnout do GitOps repozitáře (Forsgren et al., 2018). V tomto scénáři GitOps nástroj (Argo/Flux) pracuje s již vygenerovanými plain manifesty. Výhodou je plná kontrola nad obsahem v Git (co je v Git, to je přesně nasazeno). Nevýhodou může být ztráta části informací – v Git vidíme jen výsledné manifesty, ne už parametry chartu.
2. **Helm jako součást GitOps operátora:** Moderní GitOps nástroje podporují nativně práci s Helmem. Argo CD umí definici aplikace nasměrovat na Helm chart (buď lokální v repozitáři, nebo vzdálený v Helm repozitáři) a sám vykoná renderování šablon při nasazení (Vinto & Soto, 2023). Flux využívá objekt HelmRelease, který specifikuje chart a values; Fluxův Helm Controller pak provede instalaci/updatu chartu v clusteru. V obou případech je výsledkem to, že GitOps operátor spravuje Helm release obdobně jako by spravoval kolekci manifestů – sleduje verzi chartu a values v Git a zajišťuje, že cluster má odpovídající release nasazen (Yuen et al., 2021).

Druhý přístup (Helm integrován do GitOps) je dnes velmi populární, neboť spojuje výhody obou světů: *opakovaně použitelné balíčky Helm* a *automatizované kontinuální nasazování GitOps*. Uživatel například pouze aktualizuje verzi Helm chartu v deklaraci (např. změní číslo verze chartu v Argo CD Application nebo Flux HelmRelease), commitne tuto změnu a GitOps nástroj zajistí provedení Helm upgradu v clusteru (Vinto & Soto, 2023). Podobně změna parametru v values.yaml v Git repozitáři povede k novému vykreslení šablon a aplikaci změn.

Příklad Helm templatingu (výňatek z Deployment šablony):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "nginx.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ include "nginx.name" . }}
  template:
    metadata:
      labels:
        app.kubernetes.io/name: {{ include "nginx.name" . }}
```

```
spec:
  containers:
    - name: {{ .Chart.Name }}-container
      image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
      ports:
        - containerPort: {{ .Values.service.port }}
```

Listing 3.2: Ukázka šablony Helm chartu

Šablona Deploymentu využívá placeholders {{ .Values... }} pro dosazení hodnot z *values.yaml*. Například .Values.replicaCount bude nahrazen konkrétním počtem replik definovaným uživatelem v konfiguraci. Výsledné vygenerované manifesty (po spuštění Helm) jsou pak standardní YAML manifesty Kubernetes. Tímto způsobem Helm umožňuje vysokou míru znovupoužitelnosti: stejná šablona se dá použít pro různé instance aplikace pouze s jinými parametry.

Helm přináší řadu výhod v kontextu Kubernetes nasazování: usnadňuje sdílení aplikací formou chartů (existují veřejné Helm repozitáře, např. ArtifactHub, kde lze najít charty pro databáze, message queue, monitoring stacky atd.), řeší závislosti mezi charty (chart může vyžadovat jiné sub-charty) a umožňuje relativně snadno provádět rolling update a rollback aplikací (jediným příkazem helm upgrade či helm rollback) (Arundel, Domingus, 2019). V prostředí více týmů může Helm fungovat tak, že platformní tým připraví obecné charty pro běžné služby a vývojáři je jen zahrnou do svých nasazení s patřičnými values – to zvyšuje konzistenci napříč projekty.

Nevýhodou Helmu je určitá komplexita spojená s templatingem – ladění chyb v šablonách může být komplikovanější, obzvlášť pokud chart obsahuje mnoho vnořených proměnných (Kim et al., 2016). Původní verze Helm (v2) měla také architektonickou nevýhodu v podobě serverové komponenty Tiller běžící v clusteru s vysokými oprávněními, což vyvolávalo bezpečnostní obavy. Helm v3 už Tiller neobsahuje a operuje klientsky (nebo v kombinaci s GitOps operátory plně v rámci jejich práv), čímž se bezpečnost zlepšila. Nicméně, práce s Helm charty klade důraz na pečlivé verzování – nekonzistence mezi verzí chartu a verzí aplikace může způsobit zmatek. Proto se doporučuje verzovací konvenci synchronizovat, případně používat *semantic versioning* a závislostní management přímo v CI (Vinto & Soto, 2023).

V GitOps přístupu se Helm stará o balení a parametrizaci, zatímco Git a GitOps operátor řeší distribuci a aplikaci. Například podle *Cloud Native DevOps with Kubernetes* je kombinace Helmu s GitOps ideální k dosažení *multi-tenancy* a *multi-environment* nasazení:

jeden chart aplikuje aplikaci na více clusterů či jmených prostorů s různými konfiguracemi, přitom všechny instance jsou spravovány jednotně skrze centrální Git repozitář (Arundel, Domingus, 2019). Tím se snižuje duplicitní práce a riziko, že dvě prostředí se konfigurují rozdílně z nedopatření.

Helm rovněž dobře spolupracuje s Terraformem: Terraform může například instalovat aplikace formou Helm chartů v průběhu provisioningu (přes *helm_release* resource). To se využívá v bootstrapping scénářích – Terraform vytvoří cluster a pak volá Helm release pro nasazení systémových komponent (Ingress controller, monitoring). Alternativně ale GitOps přístup říká: nechejme i toto na GitOps operátorovi, ať Terraform pouze připraví cluster a vloží do něj definici (manifest) HelmRelease, o který se postará Flux. Oba modely mají opodstatnění a volba závisí na zralosti týmu s danými nástroji.

Z hlediska DevOps kultury Helm přináší výhodu sdílení a spolupráce – ops týmy mohou v Helmu poskytnout "self-service" platformu (např. chart pro nasazení standardizované aplikace), kterou vývojáři sami obsluhují skrze GitOps pipeline (Davis, Daniels, 2016). To podporuje cíl DevOps *"empowerment"* – týmy jsou schopné samy obsloužit své požadavky na infrastrukturu či middleware bez potřeby ručních zásahů centrálního týmu, což zvyšuje rychlost a snižuje fronty čekání (Forsgren et al., 2018).

Na závěr můžeme porovnat roli Helmu v GitOps s jinými přístupy ke správě konfigurace: alternativou k Helmu je například Kustomize (který místo šablon umožňuje překrývat báze manifestů tzv. patchemi) nebo Jsonnet/YAML templating. Každý přístup má své příznivce, ale Helm se stal de-facto standardem zejména díky široké knihovně chartů a známému paradigmatu správy verzí balíčků (Yuen et al., 2021). V prostředí popsaném v této práci (GitOps pipeline) Helm doplňuje ekosystém jako komponenta pro efektivní definici a údržbu aplikačních konfigurací, což přispívá k celkové efektivnosti a spolehlivosti nasazovacího procesu.

Porovnání nástrojů Argo CD, Flux, Terraform a Helm: Následující tabulka (Tab. 3.1) shrnuje klíčové vlastnosti, výhody a nevýhody všech čtyř popsaných nástrojů v kontextu jejich využití v GitOps a DevOps praxi.

Tabulka 3.1: Srovnání nástrojů Argo CD, Flux, Terraform a Helm z hlediska jejich role v GitOps a DevOps

Nástroj	Hlavní účel a charakteristika	Výhody (Pros)	Nevýhody (Cons)
Argo CD	GitOps CD operátor pro Kubernetes; běží v clusteru, synchronizuje deklarovaný stav z Git do clusteru; poskytuje UI.	– Kompletní řešení s UI, vhodné pro multi-app a multi-cluster scénáře – Podpora více typů manifestů (Helm, Kustomize…) – Pokročilé funkce: audit log, SSO, RBAC, rollbacky	– Vyšší nároky na prostředky (další služby v clusteru) – Strmější křivka učení (specifické koncepty jako Application, Project) – Bez nativní image automation (nutno doplnit další nástroj)
Flux	GitOps CD operátor (toolkit) pro Kubernetes; modulární složení z kontrolérů; konfigurace přes Kubernetes CRDs, bez nativního UI.	– Lehký, Kubernetes-native (konfigurace jako kód v rámci clusteru) – Vysoce flexibilní a rozšiřitelný (volitelné moduly) – Vestavěná image update automatizace (sledování registry)	– Chybí vestavěné UI (nutno použít CLI nebo doplňkové nástroje) – Složitější počáteční nastavení (více CRD objektů k definování) – Méně „out-of-the-box“ – vyžaduje integraci pro enterprise featury (přístupová práva apod.)
Terraform	Nástroj pro Infrastructure as Code; deklarativní popis a provisioning infrastruktury (cloud, on-prem); režim plán->aplikace.	– Multi-cloud unifikace správy (jednotný jazyk pro různé poskytovatele) – Deklarativnost a idempotence – opakovatelné zřízení prostředí – State umožňuje detekci a plánování změn před aplikací (bezpečnost změn)	– Potřeba správy stavu (state file, uzamykání, konzistence) – Není kontinuální – změny se aplikují na vyžádání, ne automat. smyčka – Složitější u driftu mimo Terraform (detekuje ale neopraví bez spuštění)
Helm	Balíčkovací a templating nástroj pro Kubernetes; umožňuje definovat aplikace jako chart s parametrickými šablonami.	– Usnadňuje sdílení a opakované použití aplikací (Charts hub, standardizace) – Možnost jednoduchého přizpůsobení nasazení pomocí values – Podpora verzování aplikací a snadný rollback/upgrady helm release	– Šablonovací syntaxe může být komplexní a hůře čitelná – Nutnost udržovat konzistenci chart vs. aplikace (versioning disciplína) – Při přímém použití helm install (mimo GitOps) možnost driftu vůči Git (řeší se integrací do GitOps nástrojů)

Zdroj: vlastní zpracování.

Argo CD a Flux zastupují kategorii GitOps operátorů pro kontinuální nasazení aplikací (CD), Terraform náleží do oblasti infrastruktury jako kód (provisioning prostředků, IaaS) a Helm funguje jako nástroj pro správu aplikačních konfigurací (balíčků). Každý z nich pokrývá odlišnou část životního cyklu – od zajištění infrastruktury (Terraform), přes definování způsobu nasazení aplikace (Helm), až po samotné provedení nasazení a jeho udržování (Argo CD/Flux). V praxi se velmi často používají společně: například Terraform připraví cloudovou infrastrukturu včetně Kubernetes clusterů, do nich se nainstaluje Argo CD nebo Flux, které pak pomocí Helm chartů nasazují konkrétní aplikace. Kombinací těchto nástrojů tak lze dosáhnout plně automatizované cesty od kódu až do produkčního prostředí (Forsgren et al., 2018). Jak uvádí Kim et al. (2016), klíčové je, že automatizace a kodifikace eliminují manuální kroky, zrychlují cyklus nasazování a zlepšují konzistenci – což jsou základní cíle DevOps i GitOps.

3.5 Odůvodnění volby technologického stacku pro praktickou část

Pro praktickou demonstraci metodiky byl zvolen technologický stack Terraform a Argo CD. Volba byla provedena s cílem zachovat vysokou míru reprodukovatelnosti testovacího prostředí, jednoznačně ověřit základní principy GitOps (Git jako jediný zdroj pravdy, auditovatelnost změn, kontinuální rekonsiliace a automatická náprava driftu) a zároveň využít nástroje, které představují současný průmyslový standard v cloud-native ekosystému (Vinto a Soto, 2023). Terraform je v tomto scénáři využit pro vrstvu IaC, tedy pro počáteční provisioning infrastruktury (Day-0 operace), zatímco Argo CD slouží jako GitOps operátor pro průběžnou synchronizaci požadovaného stavu aplikační vrstvy v rámci Kubernetes.

Alternativy existují jak ve vrstvě aplikačního GitOps, tak ve vrstvě správy infrastruktury. Pro účely této práce však nebyly zvoleny především z důvodu definovaného rozsahu a s cílem udržet demonstrační scénář co nejpřehlednější, snadno opakovatelný a striktně zaměřený na ověření samotné metodiky.

Jako alternativy k nástroji Argo CD (GitOps pro Kubernetes) lze uvažovat Flux CD, případně komplexní platformní varianty typu OpenShift GitOps. V oblasti IaC se jako alternativy k Terraformu nabízejí nástroje jako Pulumi (které definuje infrastrukturu imperativně pomocí běžných programovacích jazyků místo deklarativního HCL) (Arundel a Domingus, 2019), Crossplane (umožňující správu infrastruktury nativně přes Kubernetes API) nebo OpenTofu, což je nedávný open-source fork Terraformu plně kompatibilní s HCL ekosystémem (Hafner, 2025). Zvolená kombinace Terraformu a Argo CD však pro účely této práce poskytuje optimální poměr mezi deklarativní čitelností kódu, širokou komunitní podporou a jasným oddělením zodpovědností.

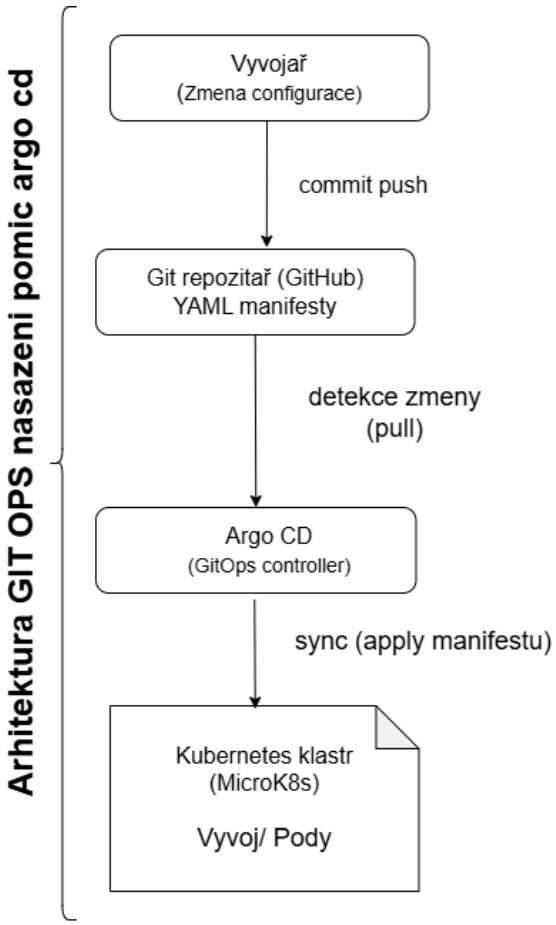
4.Implementace GitOps v cloudovém prostředí

Cílem této kapitoly je prakticky demonstrovat nasazení GitOps metodiky pro správu cloudové infrastruktury. Na základě dříve popsaných teoretických konceptů je v této části detailně popsán a proveden proces implementace GitOps s využitím nástroje Argo CD, Kubernetes a verzovacího systému Git.

Z metodologického hlediska má tato praktická část charakter Proof of Concept (PoC) a využívá metody empirického pozorování. Namísto kvantitativního měření výkonnostních metrik je kladen důraz na kvalitativní ověření správného chování systému – konkrétně na to, zda zvolený stack v reálných situacích prokazatelně naplňuje teoretické principy GitOps, jako je deklarativní synchronizace a automatická náprava driftu (self-healing)

Celý postup probíhá ve virtuálním prostředí Ubuntu 22.04 LTS, které bylo zřízeno na platformě VirtualBox. Konfigurace, nasazení a testovací aplikace jsou spravovány výhradně prostřednictvím verzovacího repozitáře Git, čímž je zajištěna deklarativní a reprodukovatelná správa infrastruktury. Kapitola je rozdělena do jednotlivých kroků – od základního nastavení systému až po plnou automatizaci nasazení aplikace prostřednictvím Argo CD.

Obrázek 4 – Architektura GitOps nasazení (GitHub → Argo CD → Kubernetes)



Zdroj: vlastní zpracování

Obrázek 4 znázorňuje základní tok GitOps nasazení použitý v této kapitole. Vývojář provádí změny konfigurace aplikace výhradně v Git repozitáři, který představuje jediný zdroj pravdy (desired state). Argo CD změny průběžně detekuje (pull) a provádí synchronizaci do Kubernetes klastru, kde jsou manifesty aplikovány a výsledný stav je průběžně porovnáván se stavem deklarovaným v Git.

Pro zajištění reprodukovatelnosti praktické části jsou v Tabulce 4.1 uvedeny parametry testovacího prostředí a použité verze klíčových nástrojů.

Tabulka 4.1 – Testovací prostředí a použité verze

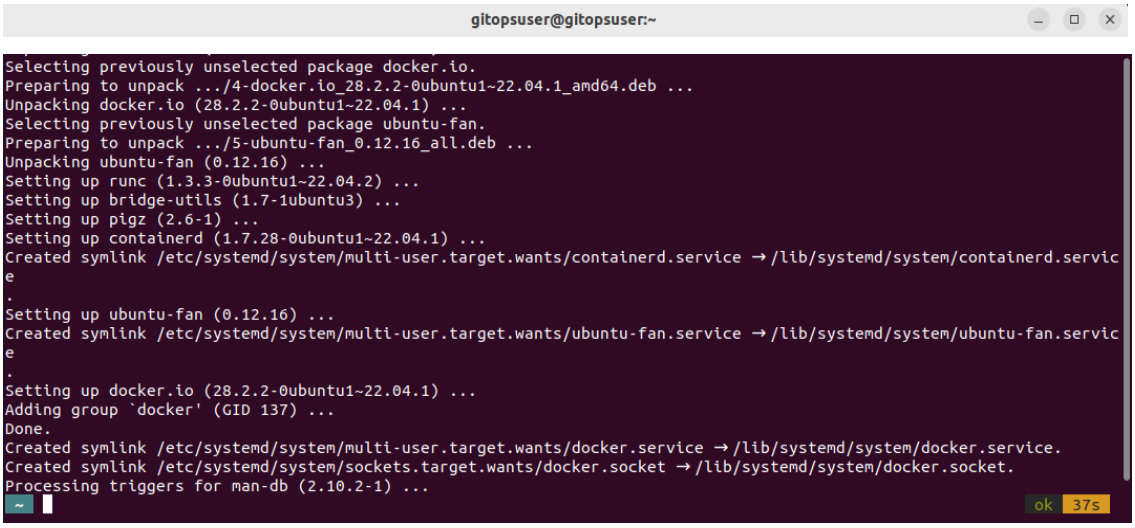
Položka	Hodnota
Virtualizační platforma	Oracle VM VirtualBox
Operační systém VM	Ubuntu 22.04 LTS (64-bit)
Parametry VM	2 CPU, 4096MB
Kubernetes distribuce	MicroK8s
Verze MicroK8s / Kubernetes	v1.32.9 revision 8511
Git	git version 2.34.1
Argo CD	v3.2.0
Přístup do clusteru	microk8s kubectl
Síťový přístup k Argo CD	port-forward (lokální přístup)

zdroj: vlastní zpracování

4.1 Proces nasazení GitOps pro správu cloudové infrastruktury

V prvním kroku byl aktualizován systém a nainstalovány základní nástroje: Git (pro práci s repozitářem), Docker (pro testovací sestavení kontejnerů) a curl (pro stahování instalačních skriptů).

Instalace proběhla úspěšně (ověřeno příkazem git --version, docker --version).



Obrázek 4.1 – Instalace nástrojů Git, Docker a curl v systému Ubuntu, zdroj: vlastní zpracování

Pro lokální nasazení Kubernetes klastra byl zvolen nástroj MicroK8s, který poskytuje minimalistickou, ale plně funkční distribuci Kubernetes vyvíjenou společností Canonical. Instalace probíhá pomocí balíčkovacího systému snap, což umožňuje rychlé zprovoznění clusteru bez nutnosti

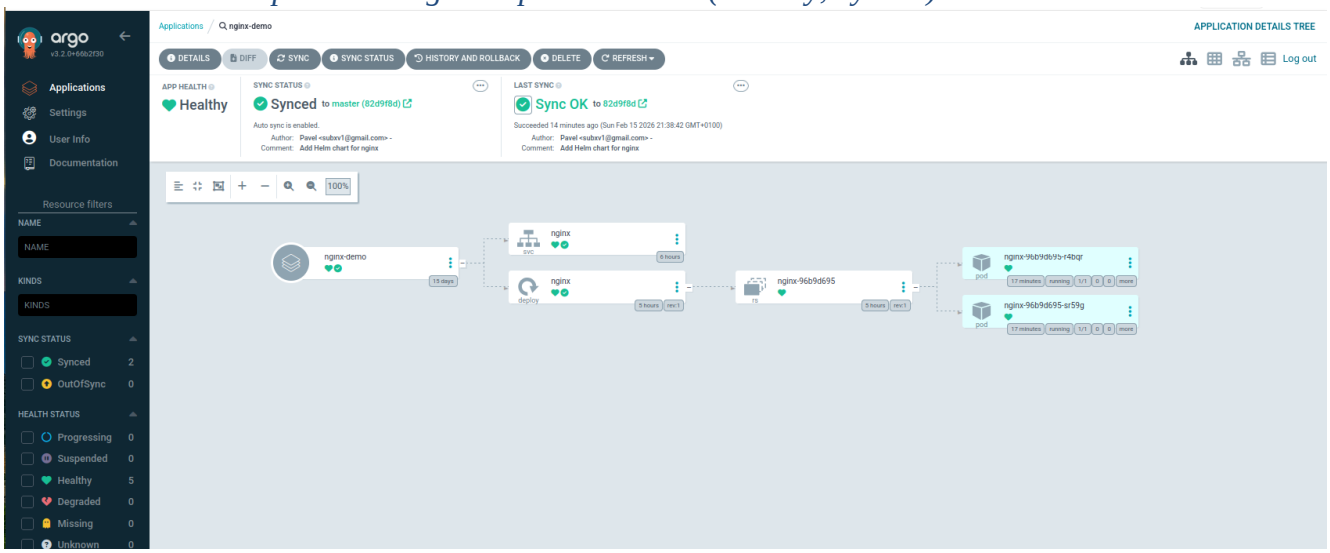
manuální konfigurace jednotlivých komponent. Po instalaci bylo nutné upravit oprávnění uživatele tak, aby bylo možné spouštět příkazy MicroK8s bez použití sudo (přidání uživatele do skupiny microk8s a obnovení relace).

GitOps vrstva byla realizována pomocí Argo CD, který průběžně sleduje stav Git repozitáře a zajišťuje synchronizaci deklarovaného stavu do cílového prostředí Kubernetes. Tím je Git repozitář využit jako jediný zdroj pravdy (desired state) a změny konfigurace aplikace jsou prováděny výhradně formou commitů.

Demonstrační aplikace nginx je v repozitáři spravována ve formě Helm chartu. Helm umožňuje parametrizaci nasazení (např. replicaCount) prostřednictvím souboru values.yaml, což je vhodné pro ukázku změnového scénáře v GitOps: úprava hodnot v Git repozitáři se po synchronizaci projeví jako řízená změna v Kubernetes clusteru.

Po dokončení uvedených kroků bylo testovací prostředí připraveno pro praktickou demonstraci GitOps – cluster byl funkční, potřebné doplňky aktivní a Argo CD dostupné pro konfiguraci aplikace. Inicializační heslo účtu admin bylo vygenerováno při instalaci Argo CD a uloženo v Kubernetes Secret argocd-initial-admin-secret; z bezpečnostních důvodů není jeho hodnota v textu práce uvedena.

Obrázek 4.2 – Stav aplikace v Argo CD po inicializaci (Healthy, Synced)



Zdroj: vlastní zpracování

4.2 Vytvoření nové aplikace v Argo CD

Po úspěšném přihlášení do webového rozhraní Argo CD byl vytvořen objekt typu Application. Tento objekt nereprezentuje „jinou“ aplikaci, ale slouží jako deklarativní popis stejné aplikace, která je definována v Git repozitáři. Jinými slovy: „aplikace v Argo CD“ a „aplikace v Git“ nejsou dvě různé aplikace – jde o jedno nasazení, kde Git obsahuje požadovaný stav (desired state)

a Argo CD tento stav kontinuálně vyhodnocuje a aplikuje do Kubernetes clusteru. Application v Argo CD tedy funguje jako konfigurační vazba mezi repozitářem (zdroj manifestů/Helm chartu), cílovým clusterem, cílovým namespace a zvolenou politikou synchronizace.

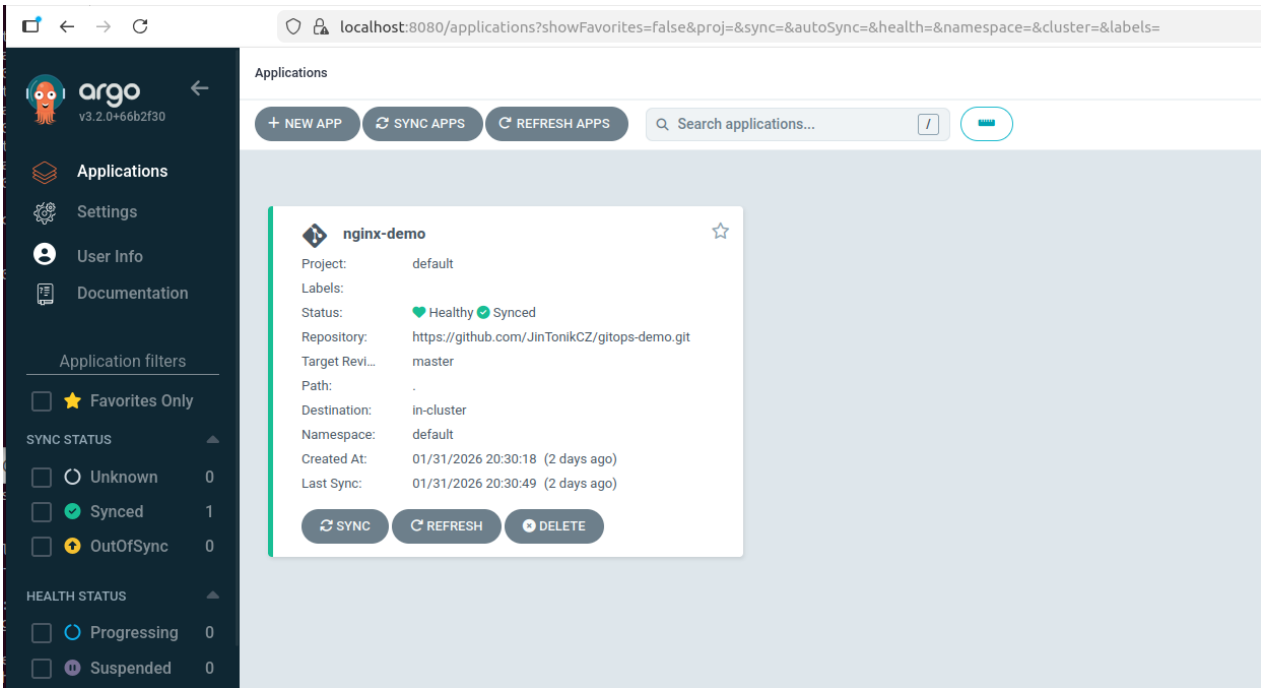
Objekt Application v Argo CD lze chápat jako „řídící záznam“ GitOps procesu: určuje odkud (repozitář + větev + cesta), kam (cluster + namespace) a jak (sync policy, případně automatizace, self-healing, prune) se má aplikace nasazovat. Díky tomu je možné jednoznačně prokázat hlavní princip GitOps, který je cílem této diplomové práce: správa infrastruktury a aplikačních konfigurací deklarativně, verzovaně a auditovatelně, přičemž nasazení je reprodukovatelné a změny jsou prováděny formou commitů v Git.

Pro praktickou demonstraci byl zvolen nginx ve formě Helm chartu, protože splňuje několik požadavků vyplývajících z cílů práce. Aplikace je záměrně jednoduchá, aby pozornost byla soustředěna na proces řízeného nasazení a na chování GitOps smyčky (detekce změny, synchronizace, self-healing), nikoliv na komplexitu samotné aplikace. Současně je nginx vhodný pro ověření změnových scénářů (např. změna replicaCount, změna Service apod.) a Helm poskytuje parametrizaci přes values.yaml, což odpovídá reálné praxi v cloud-native prostředí.

Celé řešení je realizováno v testovacím prostředí na domácím PC formou simulace cloudové infrastruktury. Namísto veřejného cloudu je použita virtualizace (VM) a Kubernetes distribuce MicroK8s, které umožňují sestavit reprezentativní „cloud-like“ prostředí: infrastruktura je definována deklarativně, řízena nástroji cloud-native ekosystému a spravována pomocí GitOps principů. Tím je naplněn praktický cíl práce – ověřit GitOps přístup na reálně použitelném stacku (Kubernetes + Argo CD + Git + Helm) a demonstrovat, že stejný postup je přenositelný i do produkčního cloudového prostředí.

Application Name: `nginx-demo`
Project: `default`
Repository URL: `https://github.com/JinTonikCZ/gitops-demo.git`
Target Revision: `master`
Path: `helm/nginx`
Cluster URL: `https://kubernetes.default.svc`
Namespace: `nginx-gitops-apps`
Sync Policy: `automated` (zapnuto `prune` a `selfHeal`)
Sync option: `CreateNamespace=true`

Obrazek 4.3 Přehled aplikací v Argo CD po vytvoření nginx-demo



Zdroj: vlastní zpracování

Po potvrzení tlačítkem Create byla aplikace zaregistrována v systému a připravena ke spuštění synchronizace.

4.2.1 Integrace GitOps s GitHub a Kubernetes klastrem

V rámci praktické části práce byla realizována integrace GitOps přístupu s nástrojem Argo CD a verzovacím systémem GitHub. Cílem této integrace bylo demonstrovat základní princip GitOps, kdy je Git repozitář považován za jediný zdroj pravdy (single source of truth) pro konfiguraci a stav aplikací běžících v Kubernetes klastru.

Byl vytvořen samostatný Git repozitář gitops-demo, který obsahuje Kubernetes manifesty popisující cílový stav aplikace. Konkrétně se jednalo o jednoduchou demonstrační aplikaci založenou na webovém serveru nginx, definovanou pomocí Deployment objektu.

Výpis 4. – Struktura repozitáře gitops-demo

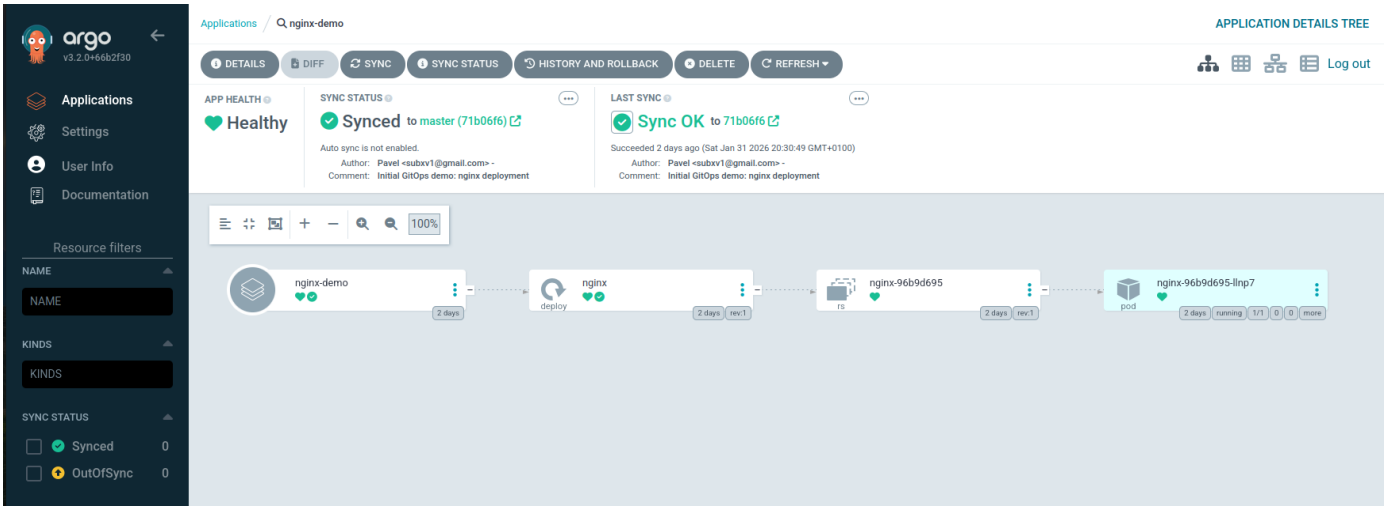
```
gitops-demo/

├── README.md
└── helm/
    └── nginx/
        ├── Chart.yaml
        ├── values.yaml
        └── templates/
            ├── deployment.yaml
            └── service.yaml
```

Repozitář obsahuje minimální sadu manifestů potřebných pro deklarativní nasazení aplikace (Deployment) a její zpřístupnění v rámci clusteru (Service).

Argo CD byl nakonfigurován tak, aby sledoval tento Git repozitář a porovnával deklarovaný stav aplikace s aktuálním stavem v Kubernetes klastru. Jakákoliv změna provedená v Git repozitáři tak automaticky představuje požadovaný stav systému, který je následně synchronizován do běhového prostředí.

Tímto způsobem byla prakticky demonstrována základní myšlenka GitOps přístupu – oddělení deklarace konfigurace aplikace od jejího nasazení a centralizace správy infrastruktury prostřednictvím verzovacího systému Git.

Obrazek 4.4 Stav aplikace nginx-demo po synchronizaci v Argo Cd



Zdroj: vlastní zpracování

4.3 Ověření GitOps přístupu pomocí změnového scénáře

V rámci demonstrace principu GitOps byla provedena změna konfigurace aplikace přímo v Git repozitáři. Konkrétně byl upraven parametr replicas v manifestu Kubernetes objektu typu Deployment, čímž došlo ke změně počtu běžících instancí aplikace nginx.

Změna byla provedena úpravou souboru nginx-deployment.yaml, kde byla hodnota parametru replicas změněna z původní hodnoty na hodnotu 2.

```
spec:
  replicas: 1  # původní stav
```

Výpis 4.3 – Původní desired state v repozitáři (nginx-deployment.yaml – výřez)

```
spec:
  replicas: 2  # nový požadovaný stav
```

Výpis 4.4 – Aktualizovaný desired state v repozitáři (nginx-deployment.yaml – výřez)

Po uložení změny vznikl nový commit v repozitáři, který byl detekován nástrojem Argo CD. Argo CD porovnal deklarovaný stav aplikace uložený v Git repozitáři s aktuálním stavem v Kubernetes klastru a aplikaci označil jako OutOfSync.

Po spuštění synchronizace (Sync) došlo k provedení změny v Kubernetes prostředí, což vedlo k vytvoření další instance aplikace nginx. Stav aplikace byl následně označen jako Synced a Healthy, čímž byla potvrzena správná funkce GitOps principu.

Tento krok prakticky demonstruje hlavní výhodu GitOps přístupu – změny konfigurace jsou prováděny výhradně prostřednictvím verzovacího systému Git a jsou reprodukovatelně aplikovány do cílového prostředí.

5. Metodika nasazeni Gitops

Předchozí kapitoly diplomové práce se zaměřovaly na teoretické vymezení konceptu GitOps, jeho principy a porovnání s tradičními přístupy ke správě cloudové infrastruktury, stejně jako na analýzu nástrojů, které GitOps podporují. Na tyto poznatky navazuje kapitola pátá, jejímž cílem je prakticky demonstrovat aplikaci GitOps metodiky a popsat jednotlivé kroky vedoucí k jejímu zavedení v reálném organizačním prostředí.

Tato kapitola je koncipována jako metodický a implementační návod, který vychází z reálných scénářů zavádění GitOps v praxi. Důraz je kladen nejen na samotnou technickou realizaci, ale také na procesní aspekty, rozhodovací body a souvislosti mezi použitými technologiemi. GitOps zde není prezentován jako izolovaný nástroj nebo technologie, ale jako ucelený přístup ke správě infrastruktury, který kombinuje principy Infrastructure as Code, verzovacího systému Git a automatizované synchronizace stavu systému.

Kapitola postupně popisuje vyhodnocení výchozího stavu organizace, návrh technologického rámce GitOps, roli Git jako stavového modelu a mechanismus reconciliation loop. Závěrem je GitOps aplikován na modelový implementační scénář a jsou diskutovány jeho přínosy, omezení a doporučení pro praktické nasazení. Cílem kapitoly je ověřit, že GitOps představuje systematický, auditovatelný a opakovatelný přístup ke správě cloudové infrastruktury.

5.1 Vyhodnocení výchozího stavu organizace z hlediska GitOps

Před samotným návrhem a implementací GitOps metodiky je nezbytné analyzovat výchozí stav organizace, zejména z hlediska používaných nástrojů, procesů a úrovně automatizace správy infrastruktury. Zavádění GitOps nelze chápat jako jednorázový technický krok, ale jako transformační proces, jehož úspěšnost je významně ovlivněna stávajícími praktikami v oblasti verzování, nasazování a správy infrastruktury.

Vyhodnocení výchozího stavu slouží k identifikaci, zda organizace již částečně využívá principy blízké GitOps, nebo zda je nutné nejprve zavést základní stavební kameny, jako je verzovací systém Git nebo deklarativní konfigurace infrastruktury. Tento krok zároveň umožňuje zvolit vhodnou strategii zavedení GitOps – postupnou standardizaci existujících procesů, nebo kompletní implementaci GitOps přístupu od základu.

Z metodického hlediska je analýza výchozího stavu založena na několika klíčových otázkách:

- zda organizace používá Git jako centrální verzovací systém,
- zda je infrastruktura spravována deklarativně pomocí Infrastructure as Code,
- zda existují kontinuální synchronizace mezi požadovaným a skutečným stavem systému,
- zda jsou změny do produkčního prostředí prováděny řízeným a auditovatelným způsobem.

Na základě odpovědí na tyto otázky lze organizaci zařadit do jedné z typických kategorií, které se v praxi při zavádění GitOps opakují.

Typické scénáře výchozího stavu

V praxi se nejčastěji setkáváme se třemi základními scénáři:

1. Organizace GitOps nevyužívá Git buď není používán vůbec, nebo slouží pouze jako úložiště aplikačního kódu bez vazby na infrastrukturu. Nasazování do clusteru probíhá manuálně nebo pomocí skriptů typu push-based deployment. Neexistuje jednotný zdroj pravdy a stav produkční infrastruktury se může od deklarace v repozitáři významně lišit.

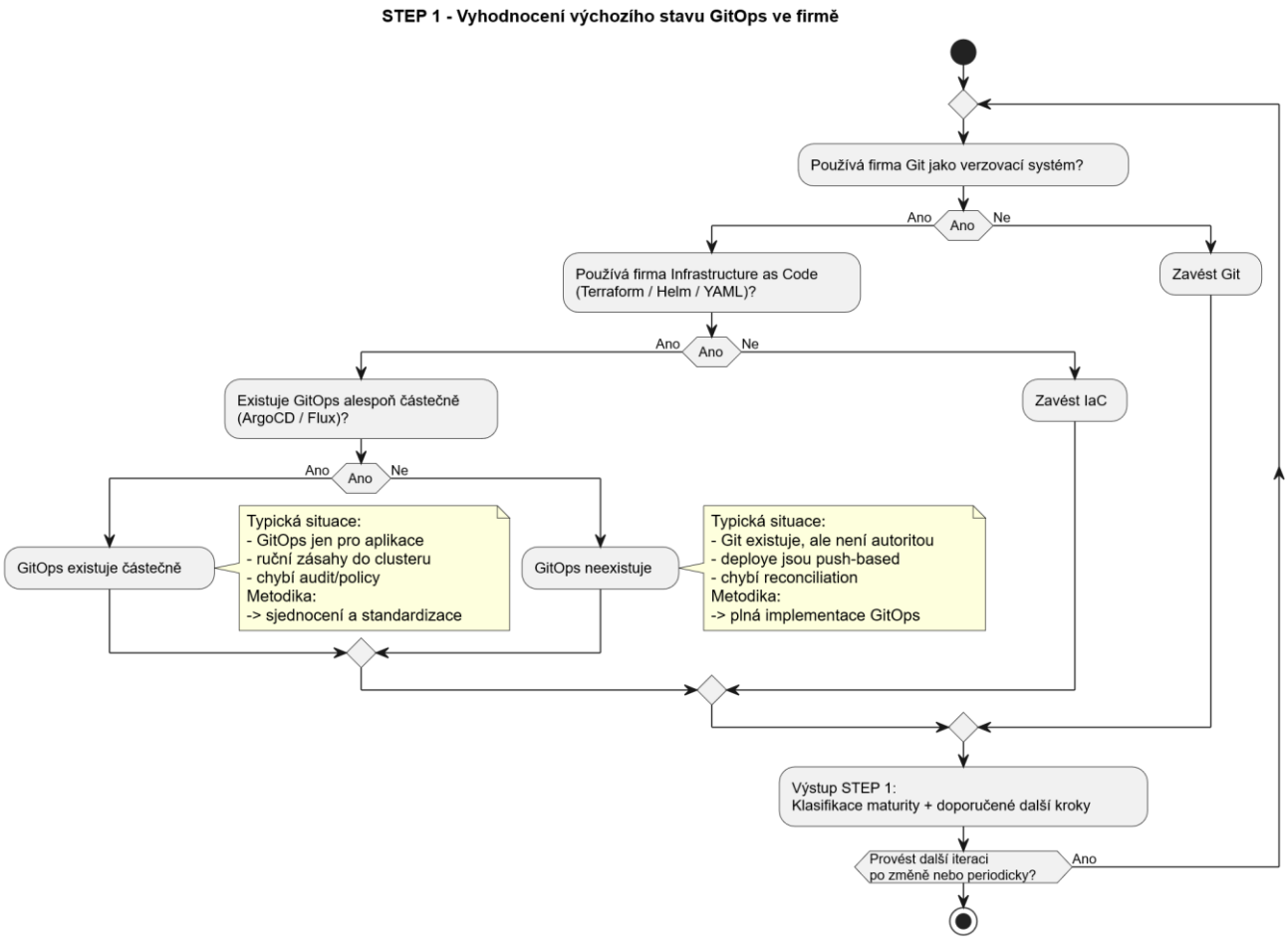
2. Organizace využívá GitOps částečně Git a Infrastructure as Code jsou používány, avšak nekonzistentně. Typickým příkladem je GitOps aplikovaný pouze na aplikační vrstvu, zatímco změny infrastruktury jsou prováděny manuálně. Chybí jednotné standardy, policy enforcement nebo centrální auditní stopa.

3. Organizace má zavedený GitOps přístup Git je využíván jako jediný zdroj pravdy, infrastruktura je definována deklarativně a změny jsou do cílového prostředí prosazovány prostřednictvím GitOps controlleru. Tento stav je však v praxi méně častý a obvykle se vyskytuje v technologicky vyspělých organizacích.

Rozhodovací model vyhodnocení připravenosti na GitOps

Pro systematické vyhodnocení výchozího stavu byl navržen rozhodovací model, který shrnuje výše uvedené otázky do přehledného schématu. Tento model umožňuje rychle určit, zda je organizace připravena na přímé zavedení GitOps, nebo zda je nejprve nutné implementovat základní podpůrné technologie a procesy.

Obrazek 5 - STEP 1: Rozhodovací model vyhodnocení výchozího stavu GitOps ve firmě



Zdroj: vlastní zpracování (PlantUML)

Rozhodovací model pracuje s postupným ověřováním klíčových předpokladů GitOps. Pokud některý z nich není splněn, model navrhuje odpovídající opatření, například zavedení verzovacího systému Git nebo Infrastructure as Code. Výstupem modelu není pouze binární rozhodnutí, ale také metodické doporučení dalšího postupu.

Význam vyhodnocení výchozího stavu

Zařazení této analytické fáze do procesu zavádění GitOps má zásadní význam. Umožňuje předejít častým chybám, kdy je GitOps implementován bez dostatečné procesní a technologické připravenosti. Zároveň poskytuje rámec pro komunikaci změn napříč týmy, jelikož jasně definuje, jaký je aktuální stav a jaké kroky je nutné podniknout pro dosažení cílového GitOps modelu.

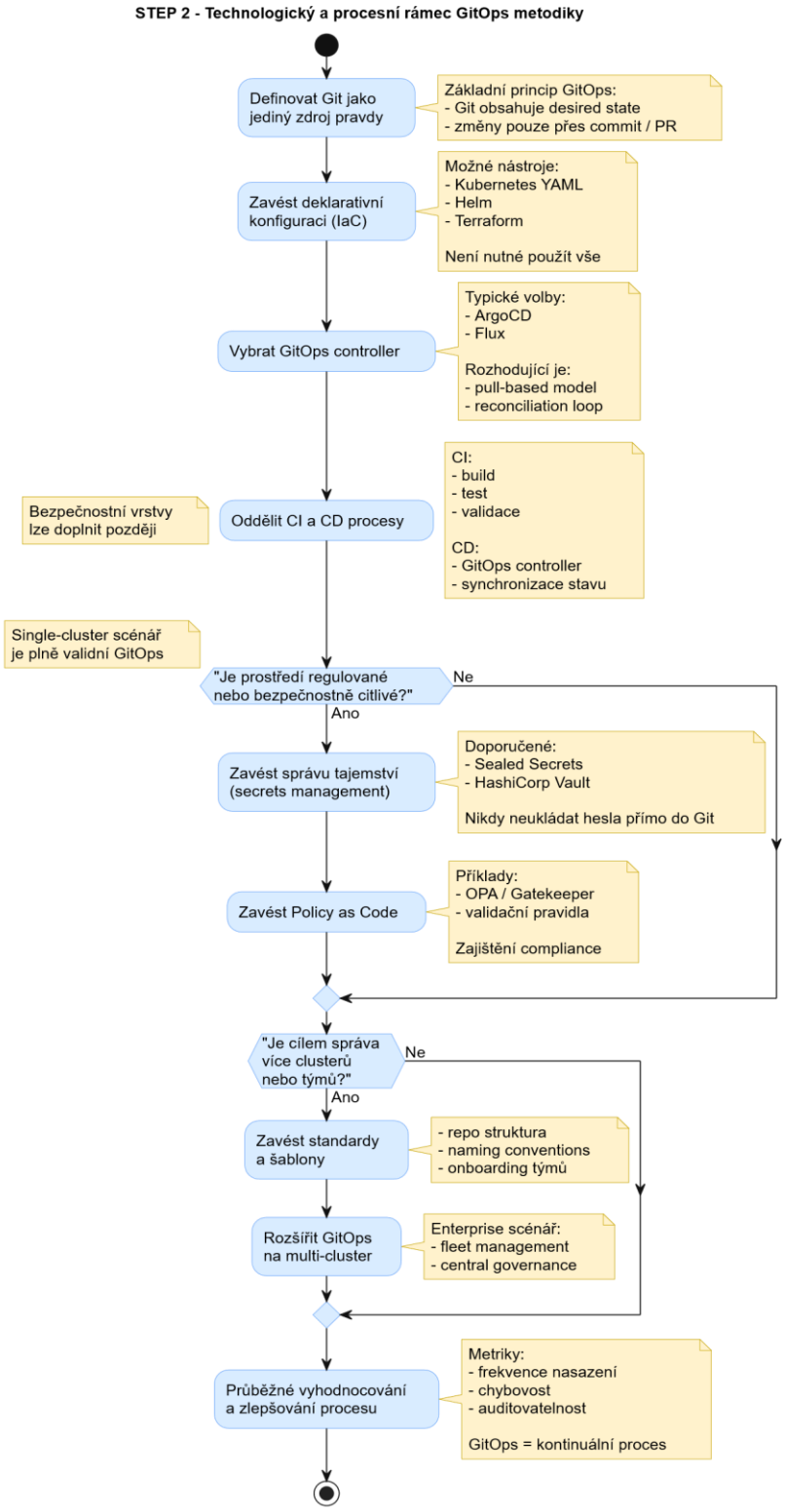
Výsledky vyhodnocení výchozího stavu slouží jako vstup pro následující kapitolu, která se zaměřuje na návrh technologického a procesního rámce GitOps metodiky.

5.2 Návrh technologického a procesního rámce zavedení GitOps

Na základě vyhodnocení výchozího stavu (STEP 1) je možné navrhnout technologický a procesní rámec zavedení GitOps. Cílem této části je popsat minimální sadu prvků, které jsou nezbytné pro funkční GitOps, a zároveň ukázat rozšiřující vrstvy, které je vhodné doplnit v případě vyšších bezpečnostních nebo organizačních požadavků.

Základní princip GitOps v této práci vychází z předpokladu, že Git repozitář obsahuje požadovaný stav systému (desired state) a jakákoliv změna prostředí je provedena formou změny v repozitáři (commit, případně PR). Distribuce změn do cílového prostředí pak probíhá pull-based způsobem, kdy GitOps controller pravidelně čte Git a prosazuje deklarovaný stav do infrastruktury. Tím je dosaženo auditovatelnosti, opakovatelnosti a snížení rizika „nezdokumentovaných“ manuálních zásahů.

Obrazek 5.1 - STEP 2: Technologický a procesní rámec GitOps metodiky



Zdroj: vlastní zpracování (PlantUML)

Schéma STEP 2 strukturuje zavedení GitOps do několika na sebe navazujících kroků:

Definovat Git jako jediný zdroj pravdy.

V první fázi je nezbytné ustanovit Git jako autoritativní úložiště konfigurace. Prakticky to znamená, že konfigurace infrastruktury a aplikací musí být uložena ve verzované podobě a změny musí procházet řízeným procesem (např. review přes pull request). Tím vzniká jednotná auditní stopa a možnost návratu k předchozímu stavu prostřednictvím verzí v repozitáři.

Zavést deklarativní konfiguraci (Infrastructure as Code).

GitOps předpokládá existenci deklarativního popisu cílového stavu. V Kubernetes prostředí se typicky jedná o YAML manifesty, případně šablony Helm nebo Kustomize. Pro cloudové zdroje mimo Kubernetes lze využít Terraform (resp. obdobné nástroje IaC). V rámci metodiky není nutné použít všechny nástroje současně – rozhodující je, aby konfigurace byla verzovatelná, čitelná a deterministicky aplikovatelná.

Vybrat GitOps controller.

Dalším krokem je volba GitOps controlleru (typicky Argo CD nebo Flux). Klíčovým kritériem výběru je podpora pull-based modelu a rekonsiliační logiky (reconciliation loop). Controller musí být schopen průběžně porovnávat deklarovaný stav v Git s reálným stavem v cílovém prostředí a v případě odchylek (drift) umožnit nápravu.

Oddělit CI a CD procesy.

V GitOps přístupu je vhodné striktně oddělit části CI a CD. CI pipeline zajišťuje sestavení artefaktů, testování a validaci (např. linting manifestů, bezpečnostní kontroly nebo kontrolu syntaktické správnosti). CD část je realizována GitOps controllerem, který synchronizuje změny do prostředí. Toto rozdělení zvyšuje přehlednost procesu a omezuje riziko, že CI přímo „píše“ do produkčního clusteru push-based způsobem.

Bezpečnostní vrstvy: správa tajemství a policy as code.

Schéma STEP 2 dále zahrnuje volitelné rozšíření o bezpečnostní vrstvy. Pokud je prostředí regulované nebo bezpečnostně citlivé, je vhodné doplnit správu tajemství (secrets management), aby

citlivé hodnoty nebyly ukládány v prostém textu do Git repozitáře. Typickými přístupy jsou šifrované sekrety (např. Sealed Secrets) nebo integrace s centrálním úložištěm tajemství (např. HashiCorp Vault).

Pro zajištění konzistence a compliance je dále možné zavést policy as code (např. OPA/Gatekeeper), které umožní validovat pravidla nasazení (omezení privilegií, povinné labely, restrikce image registrů apod.). Tím se GitOps rozšiřuje z pouhé synchronizace stavu na řízený model správy s definovanými pravidly.

Rozšíření na více týmů a clusterů.

V jednodušším scénáři (single-cluster) je GitOps plně validní i bez komplexních enterprise mechanismů. Pokud je však cílem správa více clusterů nebo týmů, je nezbytné zavést standardy a šablony: konvence pro strukturu repozitářů, naming conventions, proces onboardingu a řízení přístupových práv. V enterprise scénáři lze uvažovat o centralizované správě a fleet managementu, kde je důležitá jednotná governance a konzistentní procesní řízení.

Výstupem STEP 2 je ucelený rámec, který určuje, jaké technologické komponenty a procesní pravidla jsou pro zavedení GitOps nezbytné a které prvky je možné doplnit v návaznosti na požadavky organizace. Tento rámec tvoří přímý základ pro STEP 3, kde je podrobně vysvětlen princip Git jako stavový model a mechanismus rekonsiliace.

5.3 Vysvětlení principu GitOps na základě stavového modelu Gitu

V předchozí části (STEP 2) byl navržen technologický a procesní rámec GitOps. Pro úplnost metodiky je však důležité vysvětlit, proč GitOps není pouze „automatizované nasazování“, ale vychází z konkrétního principu správy stavu systému. Git v tomto kontextu nefunguje jen jako úložiště souborů, ale jako databáze verzovaných snapshotů, které reprezentují požadovaný stav (desired state) infrastruktury a aplikací.

STEP 3 proto vysvětluje metodické propojení interního modelu Git (stavové snímky, determinismus, auditní stopa) s GitOps controllerem, který tyto snímky interpretuje jako požadovaný stav systému a průběžně je porovnává se skutečností (actual state) v cílovém prostředí. Tím vzniká uzavřený regulační mechanismus, který je základem stabilního a auditovatelného provozu.

5.3.1 Git jako stavový model a zdroj pravdy

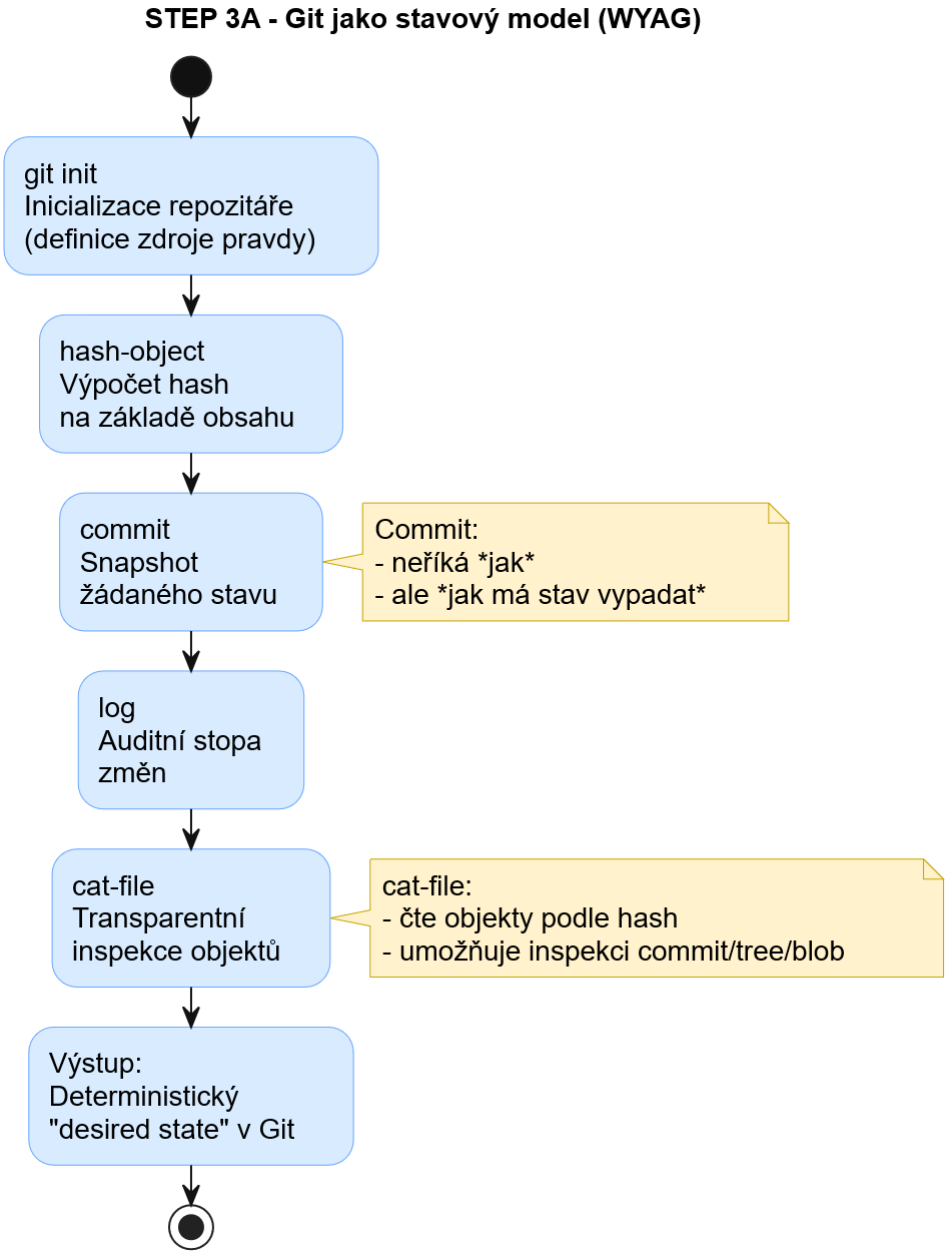
GitOps stojí na předpokladu, že konfigurace systému je vyjádřena deklarativně a je uložena ve verzovací historii. Z pohledu interního fungování Git lze commit chápat jako **snapshot** – stavový snímek obsahu, který je jednoznačně identifikován hash hodnotou a je deterministicky odvozen ze svého obsahu. To má v kontextu infrastruktury zásadní důsledky: commit nepopisuje „jak změnu provést“, ale „jak má systém v daném okamžiku vypadat“.

Pro plné pochopení tohoto mechanismu je užitečné nahlédnout do vnitřní architektury Gitu. Tuto problematiku detailně rozebírá projekt WYAG (Write Yourself a Git), jehož autorem je Thibault Polge (Polge.,2023). Polge na praktické implementaci vlastního Git klienta demonstruje, že Git nefunguje pouze jako nástroj pro sledování změn, ale primárně jako obsahově adresovatelný souborový systém (content-addressable filesystem) a striktní objektový model. V tomto modelu platí, že:

- změna obsahu vede ke změně hash hodnoty (SHA-1),
- commit reprezentuje konzistentní snímek (snapshot) stromu souborů v daném čase,
- historie commitů tvoří nezměnitelnou auditní stopu a umožňuje bezpečný návrat k předchozí verzi

Pro GitOps je tento stavový model klíčový, protože umožňuje interpretovat Git repozitář jako autoritativní zdroj definice požadovaného stavu (desired state). V praxi to znamená, že požadovaný stav je vždy jednoznačně určen konkrétní verzí repozitáře (např. konkrétním commitem nebo tagem) a je možné jej kdykoliv dohledat, porovnat a reprodukovat.

Obrazek 5.2 - STEP 3A: Git jako stavový model (WYAG)



Zdroj: vlastní zpracování (PlantUML)

Výpis 5.1– Výstup příkazů Git demonstrující stavový model (repozitář `gitops-demo`)

$ git log --oneline -3

63155a0 Increase nginx replicas from 1 to 2

$ git show --stat HEAD

nginx-deployment.yaml | 2 +-

1 file changed, 1 insertion(+), 1 deletion(-)

Praktická demonstrace stavového charakteru Git byla provedena na repozitáři gitops-demo, ve

kterém je uložen deklarativní popis požadovaného stavu testovací aplikace. Změna konfigurace byla realizována úpravou souboru nginx-deployment.yaml a následným vytvořením nového commitu. Výpis 5.X ukazuje, že commit reprezentuje jednoznačně identifikovatelný stavový snímek (snapshot) konfigurace. Historie commitů tak tvoří auditní stopu změn a umožňuje zpětně určit, jaký „desired state“ byl v daném okamžiku definován.

Commit představuje deterministický snapshot obsahu, což umožňuje jednoznačnou identifikaci stavu pomocí hash hodnoty.

Tento definovaný „desired state“ následně GitOps controller porovnává se skutečným stavem v clusteru a v rámci rekonsiliace prosazuje shodu.

5.3.2 Průběžná rekonsiliace stavu v GitOps

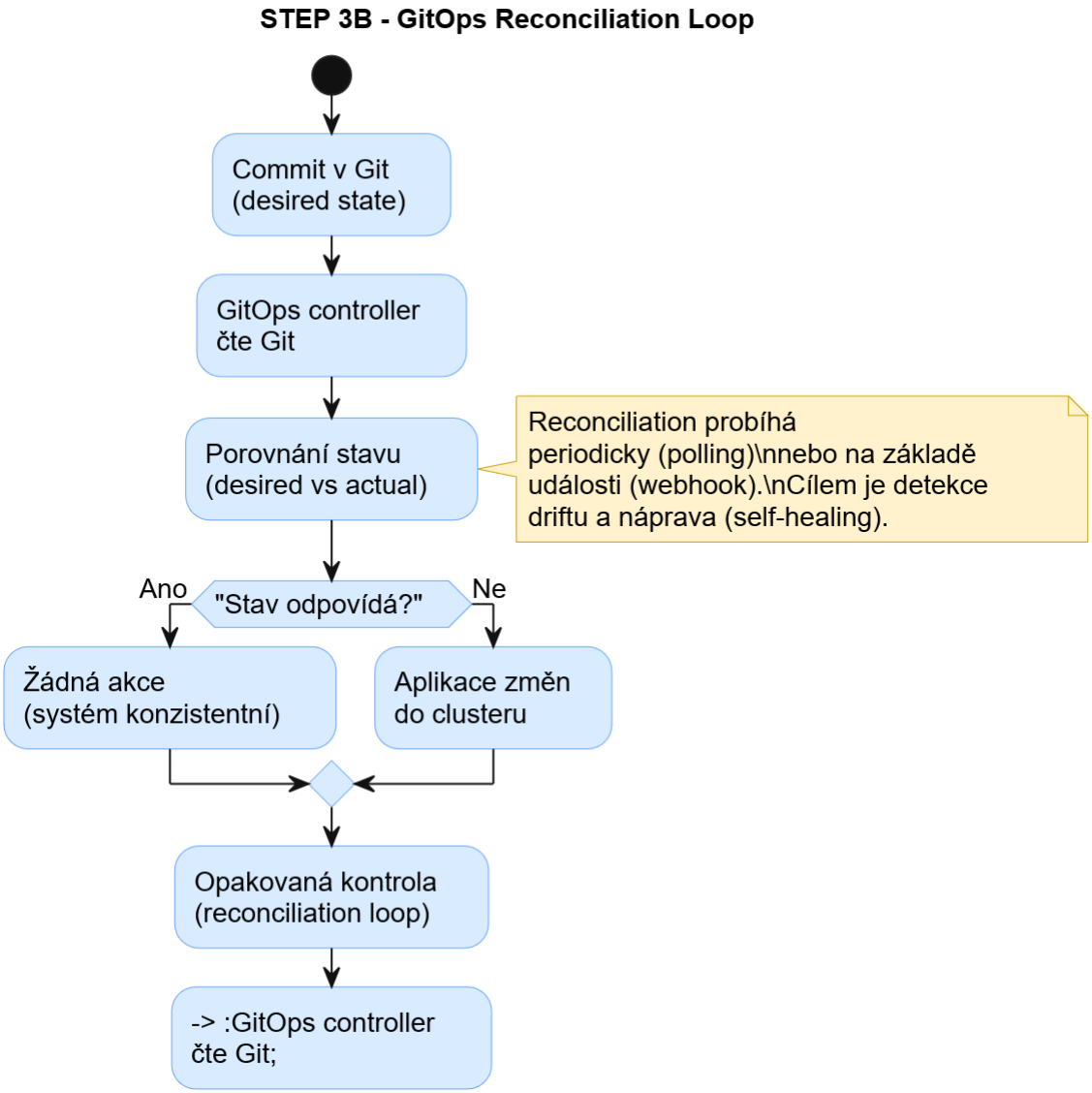
Zatímco předchozí část vysvětlila, proč je Git vhodný pro reprezentaci požadovaného stavu, GitOps přístup vyžaduje ještě mechanismus, který tento stav prosadí do cílové infrastruktury a zároveň zajistí dlouhodobou konzistenci. Tímto mechanismem je tzv. reconciliation loop.

Reconciliation loop je periodický proces, ve kterém GitOps controller:

1. načte deklarovaný stav z Git repozitáře (desired state),
2. získá aktuální stav prostředků v cílovém prostředí (actual state),
3. porovná oba stavy a určí odchylky,
4. v případě rozdílu provede synchronizaci (aplikaci změn) tak, aby actual state odpovídal desired state,
5. proces opakuje v čase a tím zajišťuje stabilitu.

Tento princip je zásadní zejména v prostředích, kde může docházet k tzv. driftu – tedy k odchylkám způsobeným manuálními zásahy, chybami nebo neočekávaným chováním systému. Reconciliation loop zde funguje jako regulační smyčka, která odchylky detekuje a umožňuje jejich nápravu bez nutnosti manuálního zásahu.

Obrazek 5.3 - STEP 3B: GitOps Reconciliation Loop



Zdroj: vlastní zpracování (PlantUML)

V praktické části (kapitola 4) byl tento princip demonstrován na změně počtu replik v manifestu, kdy změna provedená commitem do Git byla GitOps controllerem automaticky aplikována do clusteru.

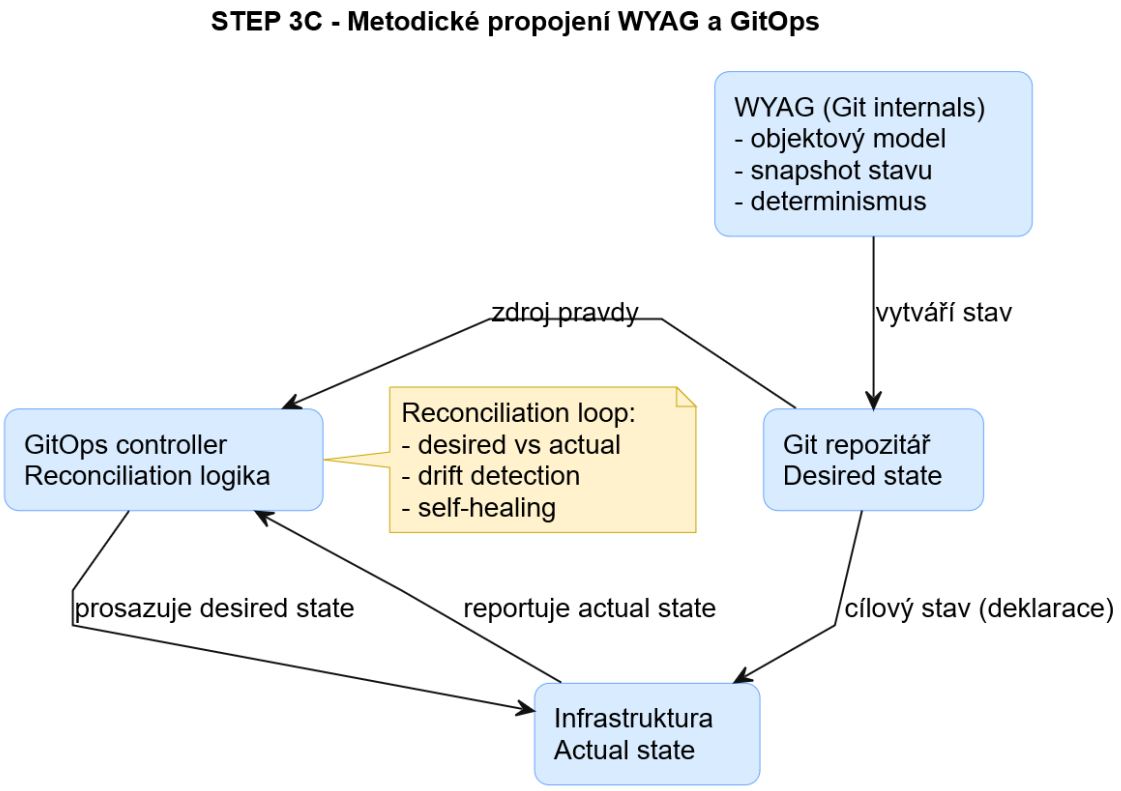
5.3.3 Vazba stavového modelu Gitu na činnost GitOps nástrojů

Propojení WYAG (stavový model Git) a GitOps (rekonsiliační logika) lze shrnout do jednoduchého vztahu: Git poskytuje **model stavu a historii změn**, GitOps controller zajišťuje **interpretaci a prosazení tohoto stavu** do cílové infrastruktury. Z metodického hlediska je důležité, že tyto dvě části řeší různé problémy:

- **Git (WYAG)** řeší otázku *„jak reprezentovat požadovaný stav a udržet auditní stopu“* → snapshoty, historie, determinismus, možnost návratu k verzi.
- **GitOps controller** řeší otázku *„jak tento stav průběžně porovnávat s realitou a prosazovat“* → reconciliation loop, drift detection, synchronizace změn.

Výsledkem je provozní model, ve kterém je Git repozitář jediným zdrojem pravdy pro deklarovaný stav a infrastruktura je neustále udržována v konzistentním stavu vůči této deklaraci. Tento model je vhodný nejen pro automatizaci nasazení, ale zejména pro auditovatelnost a standardizaci změn – každá změna je dohledatelná v historii commitů a její dopad lze zpětně analyzovat

STEP 3C: Metodické propojení WYAG a GitOps



Zdroj: vlastní zpracování (PlantUML)

Na základě STEP 3 lze GitOps chápat jako systematický způsob řízení infrastruktury, kde je změna konfigurace vždy reprezentována změnou ve verzované definici (Git) a cílové prostředí je vůči této definici průběžně synchronizováno (GitOps controller). Tím je vytvořen jednotný a opakovatelný proces, který je přenositelný do různých organizačních a technologických kontextů.

5.4 Rozšíření metodiky o ověření odolnosti a enterprise přesah (STEP 4 a STEP 5)

Zatímco kroky STEP 1 až 3 definují samotné jádro zavedení GitOps (příprava, nasazení, rekonsiliace), pro komplexní ověření metodiky v praxi jsou nezbytné další dva kroky, které simulují reálný provoz a plánování budoucího rozvoje:

- **STEP 4 – Ověření self-healing mechanismu (náprava driftu):** Metodický krok zaměřený na testování odolnosti nasazeného řešení. Zahrnuje umělé vyvolání chyby (driftu) mimo Git a ověření, zda GitOps controller dokáže autonomně uvést systém zpět do požadovaného stavu.
- **STEP 5 – Vyhodnocení a doporučení pro produkční nasazení:** Krok, který přesahuje základní implementaci a definuje, jak řešení integrovat s podnikovými standardy (governance, bezpečnost, oddělení repozitářů). Tyto dva kroky ucelují metodický rámec a jsou prakticky demonstrovány v kapitole 6.

5.5 Návaznost metodiky na praktickou implementaci (kapitola 4)

Tato část shrnuje hlavní výstupy praktické implementace popsané v kapitole 4 a navazuje na metodický rámec definovaný v kapitolách 5.1–5.3 (STEP 1–3). Cílem není opakovat postup instalace a konfigurace, ale jednoznačně vymezit, jaké artefakty vznikly a jaké vlastnosti GitOps přístupu byly prakticky ověřeny.

5.5.1 Vytvořené artefakty a jejich role

Výstupem praktické části je sada technických a dokumentačních artefaktů, které dohromady tvoří reprodukovatelný příklad GitOps přístupu v Kubernetes prostředí:

- **Testovací prostředí a verze nástrojů (Tabulka 4.1)**

Umožňuje reprodukovatelnost postupu, protože definuje parametry prostředí (VM/OS) a použité verze klíčových komponent (MicroK8s/Kubernetes, Argo CD, Git).

- **Git repozitář gitops-demo jako jediný zdroj pravdy**

Repozitář obsahuje deklarativní konfiguraci aplikace (např. nginx-deployment.yaml, případně doplněnou o nginx-service.yaml) a dokumentační soubor README.md. Struktura repozitáře je uvedena ve Výpisu 4.1.

- **Konfigurace GitOps controlleru (Argo CD) a jeho vazba na repozitář**

Byla vytvořena vazba mezi Argo CD a Git repozitářem tak, aby Argo CD průběžně četl deklarovaný stav a prosazoval jej do clusteru. Stav synchronizace byl ověřen ve webovém rozhraní Argo CD (Obrázek 4.2) a pomocí výpisů z clusteru (Výpisy 4.3 a 4.4).

- **Doklad o stavovém modelu Git (Výpis 5.1)**

Bylo doloženo, že změny konfigurace jsou reprezentovány jako nové commity v Git, čímž vzniká auditní stopa změn a jednoznačné určení „desired state“ v konkrétní verzi repozitáře.

5.5.2 Ověřené vlastnosti GitOps přístupu

Na základě provedené implementace lze shrnout následující prakticky ověřené vlastnosti GitOps:

1. **Deklarativní definice požadovaného stavu (desired state)**

Konfigurace aplikace je definována deklarativně v repozitáři, což umožňuje snadnou kontrolu a opakovatelné nasazení.

2. **Auditovatelnost změn prostřednictvím verzování**

Každá změna konfigurace je dohledatelná v historii repozitáře, včetně informace o tom, kdy a jaká změna byla provedena. To podporuje transparentnost a zjednodušuje zpětnou analýzu.

3. **Pull-based model a oddělení CI/CD odpovědností**

Distribuce konfigurace do clusteru je realizována GitOps controllerem, který čte Git jako autoritativní zdroj. Tento přístup snižuje závislost na push-based skriptech a omezuje riziko přímých změn v cílovém prostředí.

4. **Rekonsiliační princip (desired vs. actual state)**

Stav aplikace v clusteru je průběžně porovnáván s definicí v Git, což vytváří konzistentní provozní model a podporuje dlouhodobou stabilitu proces.

5.5.3 Stručné zhodnocení dosaženého výsledku

Praktická implementace potvrdila, že i v jednoduchém laboratorním scénáři lze GitOps metodiku aplikovat způsobem, který zachovává její základní principy: Git jako jediný zdroj pravdy, deklarativní konfigurace a průběžná synchronizace stavu prostřednictvím GitOps controlleru. Z metodického hlediska je důležité, že demonstrace nebyla provedena jako jednorázový „deployment“, ale jako řízený proces změn, kde je konfigurace systému reprezentována verzovaným stavem a aplikována do cílového prostředí pull-based způsobem.

6. Výsledky a diskuze

6.1 Realizace metodiky Step 1–5 v cloudovém prostředí

6.1.1 Step 1 – Vyhodnocení výchozího stavu

Na základě rozhodovacího modelu uvedeného v kapitole 5 (STEP 1) byl testovací cloud klasifikován jako greenfield prostředí (tj. nové prostředí bez existujícího GitOps řešení a bez zavedeného GitOps controlleru). Následující ověření vychází z metodiky popsané v kapitole 5 a je realizováno v prostředí definovaném v kapitole 4. Cílem kroků Step 1–5 je zavést pull-based GitOps řízení, kde Git slouží jako jediný zdroj pravdy (desired state) a Argo CD průběžně zajišťuje synchronizaci tohoto stavu do prostředí Kubernetes.

Kroky Step 4 a Step 5 navazují na jádro metodiky (STEP 1–3) a rozšiřují praktické ověření o typické provozní vlastnosti GitOps (self-healing) a stručné zhodnocení rozšiřitelných oblastí (governance, bezpečnost, Terraform).

Výstupem kroku Step 1 je klasifikace prostředí jako greenfield a stanovení cíle ověření: zavést Git jako „single source of truth“ a ověřit reconciliation mechanismus GitOps controlleru.

6.1.2 Step 2 – Založení GitOps controlleru a propojení s Git (Argo CD)

V rámci kroku Step 2 byl v prostředí zaveden GitOps controller Argo CD (kap. 4) a byla vytvořena Argo CD Application, která reprezentuje stejnou aplikaci jako v Git repozitáři. „Application“ v Argo CD zde neznamená jiný software, ale konfigurační objekt, který definuje vazbu na konkrétní repozitář, větev a cestu (Helm chart). Argo CD byl zvolen jako GitOps controller díky nativní podpoře Kubernetes, přehlednému UI pro sledování stavu aplikace a možnosti automatické synchronizace v režimu pull-based. V Kubernetes se tato vazba projeví jako objekt spravovaný Argo CD, který definuje zdroj (repozitář), cílový cluster/namespace a způsob nasazení (Helm). Tímto je zajištěno, že veškeré změny jsou řízeny přes Git a Argo CD je pouze vykonavatel synchronizace (pull model). Od tohoto okamžiku Argo CD průběžně porovnává deklarovaný stav (Git, desired state) se skutečným stavem v Kubernetes a zajišťuje synchronizaci.

Pro demonstrační scénář byl zvolen Helm chart nginx uložený v repozitáři, protože umožňuje jednoduchou parametrickou změnu (replicaCount) a jasně pozorovatelný dopad do clusteru.

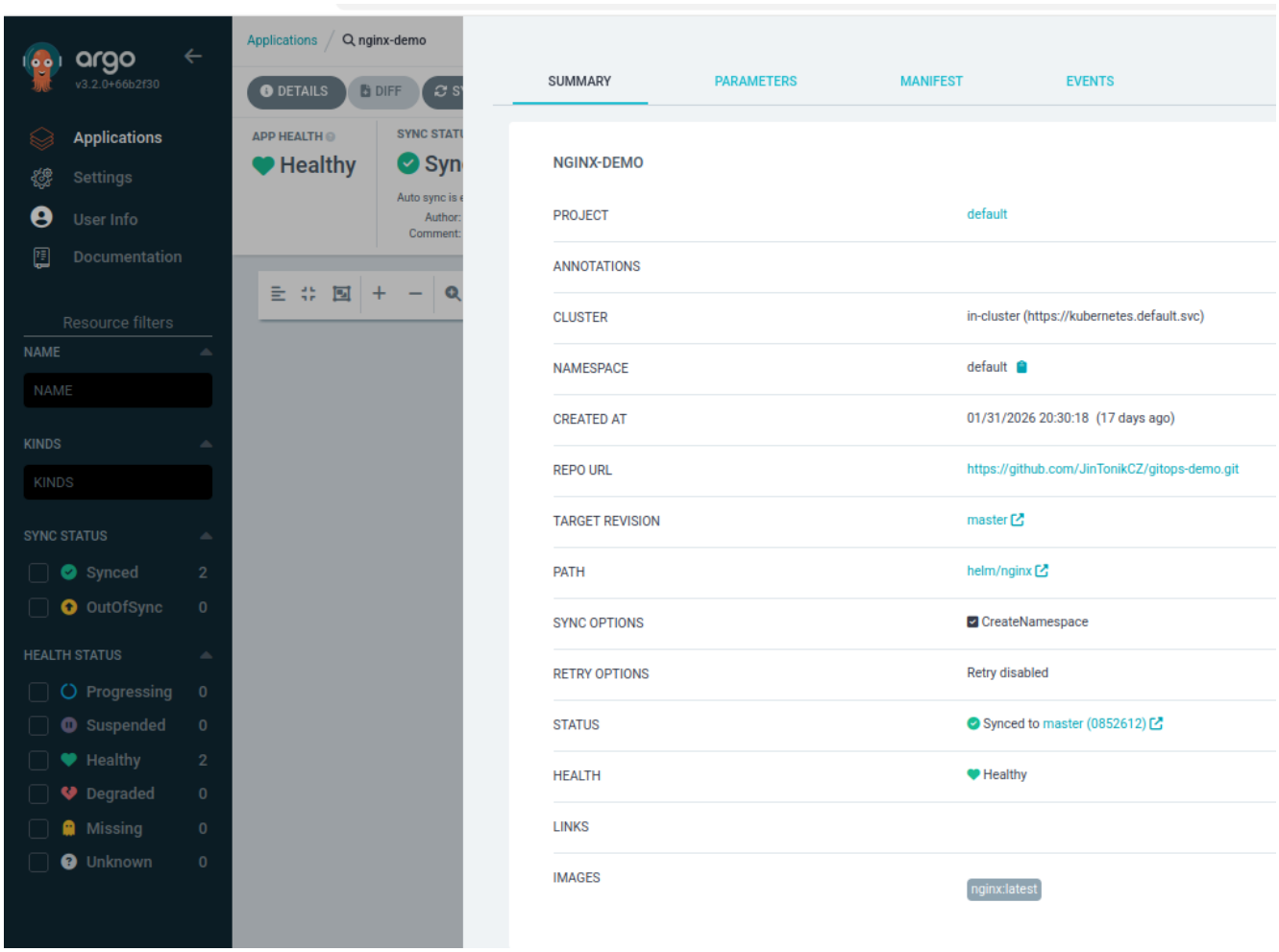
Scénář je zvolen jako minimální reprodukovatelný příklad správy cloud-native komponent v Kubernetes (deployment, service a jejich životní cyklus), který odpovídá cíli práce: demonstrovat řízení požadovaného stavu infrastruktury/aplikace pomocí GitOps.

Vytvořená aplikace *nginx-demo* byla navázána na:

- repozitář: https://github.com/JinTonikCZ/gitops-demo

- větev: master
- cesta k definici: helm/nginx (Helm chart)
- cílový namespace: default
- synchronizace: *automated* (prune + selfHeal)

Tímto krokem byla vytvořena vazba mezi Git repozitářem a clusterem; od této chvíle Argo CD průběžně vyhodnocuje rozdíly a provádí synchronizaci podle zvolené politiky.



Obrázek 6.1: Stav aplikace nginx-demo v Argo CD (Healthy, Synced) a strom nasazených Kubernetes objektů. Zdroj: vlastní zpracování.

6.1.3 Step 3 – Git jako jediný zdroj pravdy a změnový scénář

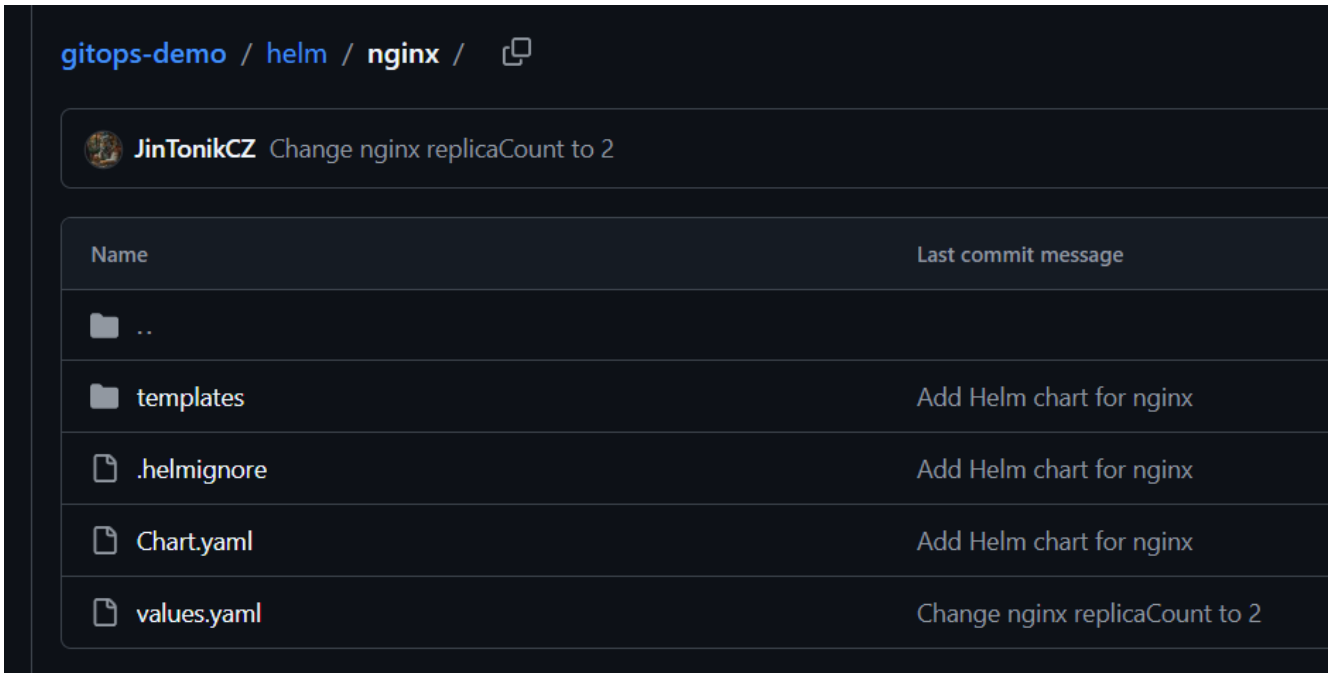
V rámci praktické realizace byl krok Step 3 rozdělen podle schémat uvedených v kapitole 5 na tři dílčí části (STEP 3A–3C). Tyto části odpovídají logice:

1. vznik nového desired state v Git
2. rekonsiliace v GitOps controlleru
3. ověření a interpretace výsledku

STEP 3A – Vznik nového „desired state“ v Git (změna v repozitáři)

V kroku Step 3 byl ověřen princip „Git jako jediný zdroj pravdy“ pomocí řízené změny konfigurace aplikace uložené v repozitáři. Změna byla provedena úpravou parametru replicaCount v souboru values.yaml Helm chartu (z hodnoty 1 na 2) a následným commitem a pushnutím změny do vzdáleného Git repozitáře.

Výstupem kroku STEP 3A je nový verzovaný stav v Git, který reprezentuje požadovanou konfiguraci aplikace.



Obrázek 6.2: Git repozitář jako jediný zdroj pravdy – Helm chart aplikace nginx (Chart.yaml, values.yaml, templates). Zdroj: vlastní zpracování.

STEP 3B – Detekce změny a rekonsiliace v Argo CD (OutOfSync → Synced)

Po publikování commitu Argo CD vyhodnotil rozdíl mezi deklarovaným stavem v Git a skutečným stavem v clusteru. Aplikace se v Argo CD dočasně zobrazila jako OutOfSync, následně byla provedena synchronizace a stav se vrátil do Synced/Healthy. Tím byl prakticky ověřen princip reconciliation loop popsaný v kapitole 5 (STEP 3B).

STEP 3C – Ověření výsledku v Kubernetes a auditní stopa změny

Výsledek byl ověřen přímo v Kubernetes, kde došlo ke škálování Deploymentu na dvě repliky

a vytvoření druhého Podu. Zároveň je změna auditovatelná ve dvou rovinách: v historii Git repozitáře (commit) a v historii synchronizací v Argo CD. Tím je potvrzena metodická vazba mezi verzovaným stavovým modelem Git a činností GitOps controlleru (STEP 3C).

```
~ kubectl get pods -n default                                          ok 7s
NAME                    READY   STATUS    RESTARTS   AGE
nginx-96b9d695-r4bqr    1/1     Running   0          52m
nginx-96b9d695-sr59g    1/1     Running   0          52m
```

Obrázek 6.3 – Ověření změny v Kubernetes: dvě repliky nginx po synchronizaci. Zdroj: vlastní zpracování.

6.1.4 Step 4 – Self-healing (drift → automatická náprava)

V rámci kroku Step 4 byla ověřena schopnost GitOps controlleru automaticky napravit odchylku (*configuration drift*) mezi deklarovaným stavem v Git a skutečným stavem v Kubernetes. Drift byl záměrně vyvolán manuální změnou v clusteru mimo Git proces.

Jak je patrné z výpisu terminálu (viz **Obrázek 6.4**), pomocí příkazu kubectl scale byl počet replik Deploymentu nginx manuálně snížen na nulu. Tím došlo k okamžitému nesouladu mezi požadovaným stavem (2 repliky v Gitu) a reálným stavem (0 replik). Protože byla na aplikaci v Argo CD aktivována politika selfHeal: true, controller tuto odchylku okamžitě detekoval.

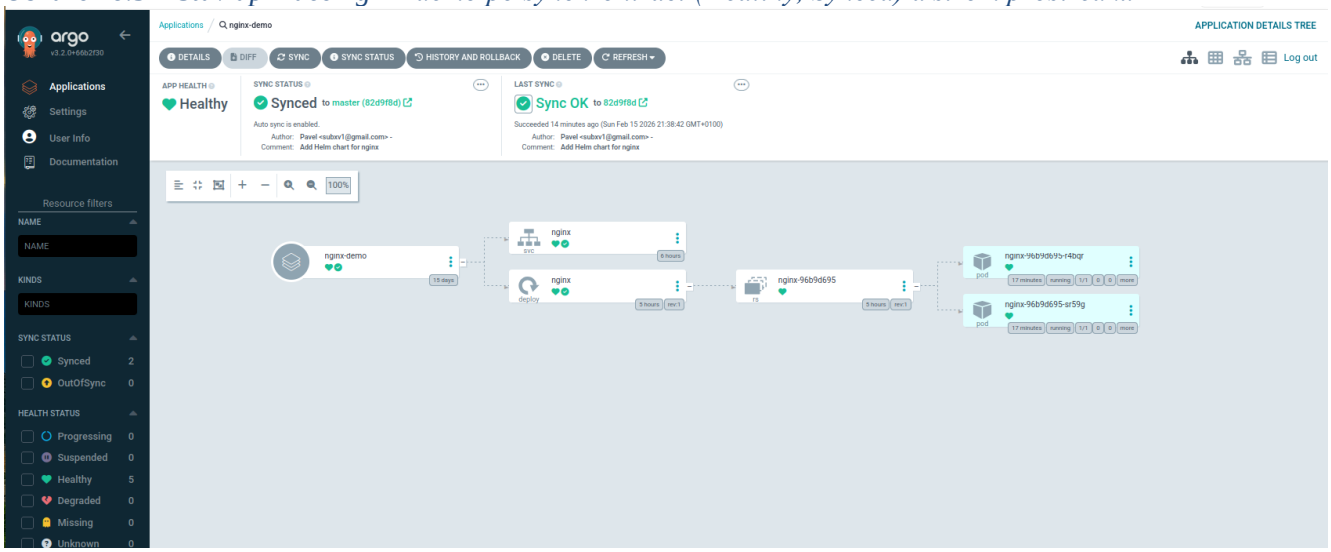
Obrázek 6.4 – Demonstrace self-healing procesu v příkazové řádce: simulace driftu (změna počtu replik na 0) a okamžitá automatická obnova podů nástrojem Argo CD.

```
gitopsuser@gitopsuser:~/gitops-demo
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   2/2     2            2           10h

^C%
 ~/gitops-demo  master  microk8s kubectl get deploy nginx -n default                                  1 err  1m 4s  %
microk8s kubectl get pods -n default | grep nginx
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   2/2     2            2           10h
nginx-96b9d695-z9mlf   1/1     Running   0          87s
nginx-96b9d695-zlxgw   1/1     Running   0          87s
 ~/gitops-demo  master  microk8s kubectl -n argocd get app nginx-demo \                                                ok | %
  -o jsonpath='{range .status.history[*]}id={.id}{"  "}deployedAt={.deployedAt}{"  "}rev={.revision}{"  "}by={.initiatedBy.us
ername}{"\n"}{end}'
id=0  deployedAt=2026-01-31T19:30:49Z  rev=71b06f68b51f7a9f8ac4d867b764f21dee36b580  by=admin
id=1  deployedAt=2026-02-02T22:07:18Z  rev=63155a07dde177f4170f0d1b52319d60f502f97f  by=admin
id=2  deployedAt=2026-02-15T15:18:33Z  rev=82d9f8d92ae76e9c4cb247b036b67e48b18013ec  by=
id=3  deployedAt=2026-02-15T15:53:24Z  rev=82d9f8d92ae76e9c4cb247b036b67e48b18013ec  by=
id=4  deployedAt=2026-02-15T20:38:42Z  rev=82d9f8d92ae76e9c4cb247b036b67e48b18013ec  by=admin
id=5  deployedAt=2026-02-15T22:15:22Z  rev=08526127b741938b364cd98e35b6ee68730ce47e  by=admin
 ~/gitops-demo  master  microk8s kubectl -n argocd get app nginx-demo \                                        ok  4s  %
  -o jsonpath='phase={.status.operationState.phase}{"  "}started={.status.operationState.startedAt}{"  "}finished={.status.op
erationState.finishedAt}{"\n"}'
phase=Succeeded  started=2026-02-16T01:57:05Z  finished=2026-02-16T01:57:09Z
 ~/gitops-demo  master  kubectl get deploy nginx -n default                                                          ok | %
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   2/2     2            2           10h
 ~/gitops-demo  master  microk8s kubectl -n argocd get app nginx-demo -o jsonpath=...operationState...               ok | %
...operationState...%
 ~/gitops-demo  master  microk8s kubectl -n argocd get app nginx-demo -o jsonpath='{.spec.syncPolicy.automated}{"\n"}'
{"enabled":true,"prune":true,"selfHeal":true}
 ~/gitops-demo  master  microk8s kubectl scale deploy nginx -n default --replicas=0                                  ok | %

deployment.apps/nginx scaled
 ~/gitops-demo  master  microk8s kubectl get deploy nginx -n default                                                 ok | %

NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   2/2     2            2           10h
 ~/gitops-demo  master  microk8s kubectl get pods -n default | grep nginx                                        ok  3s  %
nginx-96b9d695-2m42g   1/1     Running   0          16s
nginx-96b9d695-fmg4x   1/1     Running   0          15s
 ~/gitops-demo  master  microk8s kubectl get deploy nginx -n default -w                                              ok | %
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   2/2     2            2           10h
^C%
 ~/gitops-demo  master  microk8s kubectl -n argocd get app nginx-demo \                                  1 err  28s  %
  -o jsonpath='phase={.status.operationState.phase} started={.status.operationState.startedAt} finished={.status.operationSta
te.finishedAt}{"\n"}'
phase=Succeeded started=2026-02-16T02:07:57Z finished=2026-02-16T02:07:58Z
 ~/gitops-demo  master                                                                                                ok | %
```

Zdroj: vlastní zpracování.

Bez jakéhokoliv dalšího manuálního zásahu vyvolal Argo CD novou synchronizační smyčku a během několika sekund obnovil běžící pody do požadovaného stavu (dvě repliky), což dokazuje i stáří nově vytvořených podů ve výpisu terminálu. Po provedení automatické nápravy se aplikace v uživatelském rozhraní Argo CD opět zobrazila v plně synchronizovaném a zdravém stavu (viz Obrázek 6.5). Tento experiment potvrdil vysokou míru odolnosti systému a schopnost automatické obnovy bez nutnosti zásahu administrátora.

Obrázek 6.5 – Stav aplikace nginx-demo po synchronizaci (Healthy, Synced) a strom prostředků



Zdroj: vlastní zpracování

6.1.5 Step 5– Shrnutí ověření a doporučení pro rozšíření (governance, bezpečnost, Terraform)

Kroky Step 1–4 ověřily základní principy GitOps v testovacím prostředí: Git jako jediný zdroj pravdy, pull-based synchronizaci pomocí Argo CD, auditovatelnost změn a automatickou nápravu driftu (self-healing). Z praktického ověření vyplývá, že zvolený scénář je vhodný jako minimální reprodukovatelný příklad pro demonstraci GitOps řízení nasazení aplikace v Kubernetes.

Pro rozšíření řešení směrem k reálnému provoznímu nasazení lze doporučit zejména:

Governance: zavedení práce přes Pull Requesty, schvalování změn a definice odpovědností (např. CODEOWNERS).

Bezpečnost: integrace správy tajemství (např. SOPS/Sealed Secrets nebo externí secret manager), omezení oprávnění (RBAC) a audit logů.

Terraform/IaC: rozšíření GitOps o správu infrastruktury (např. Terraform pro cloudové zdroje) a jasné oddělení repozitářů pro infrastrukturu a aplikace.

6.2 Pozorované výsledky

Na základě realizace metodiky STEP 1–5 (kap. 5) byly v testovacím prostředí (kap. 4) pozorovány následující výsledky. Výsledky jsou interpretovány ve vazbě na ověřované vlastnosti GitOps: Git jako jediný zdroj pravdy, průběžná rekonsiliace (reconciliation loop), auditovatelnost změn a self-healing (náprava driftu).

Detekce změn a synchronizace: Argo CD spolehlivě detekoval změny v repozitáři a přechod aplikace do stavu OutOfSync odpovídal očekávanému chování. Následná synchronizace vedla k návratu do stavu Synced/Healthy. V laboratorních podmínkách se odezva na změnu pohybovala typicky v řádu desítek sekund (v závislosti na intervalech kontroly, dostupných prostředcích a síťových podmínkách).

Deklarativní změna desired state (škálování): Změna parametru replicaCount v Helm chartu (1 → 2) vedla k vytvoření druhé repliky Deploymentu bez nutnosti manuálního zásahu do clusteru. Tím byl ověřen princip, že jediným „spouštěčem“ změny je commit v Git repozitáři a GitOps controller vykonává pouze synchronizaci deklarovaného stavu.

Self-healing a náprava driftu: Po záměrném vyvolání driftu mimo Git (manuální změna počtu replik) Argo CD odchylku detekoval a automaticky obnovil stav dle deklarace v Git. Tento výsledek potvrzuje praktickou použitelnost GitOps pro minimalizaci konfiguračního driftu a pro zvýšení provozní odolnosti (zejména v režimu selfHeal).

Souhrnně lze konstatovat, že byly prakticky ověřeny klíčové vlastnosti GitOps definované v metodice:

- Git jako single source of truth
- průběžná rekonsiliace
- auditovatelnost změn
- automatická náprava driftu.

6.3 Diskuze, limity a doporučení

Praktická implementace potvrdila, že GitOps výrazně zvyšuje spolehlivost a auditovatelnost správy infrastruktury. Přesto nasazení tohoto přístupu v produkčním prostředí přináší specifické výzvy, zejména v oblasti bezpečnosti a komplexní správy cloudových zdrojů, které je nutné v rámci diskuze reflektovat.

Bezpečnost a správa tajemství (Secrets Management)

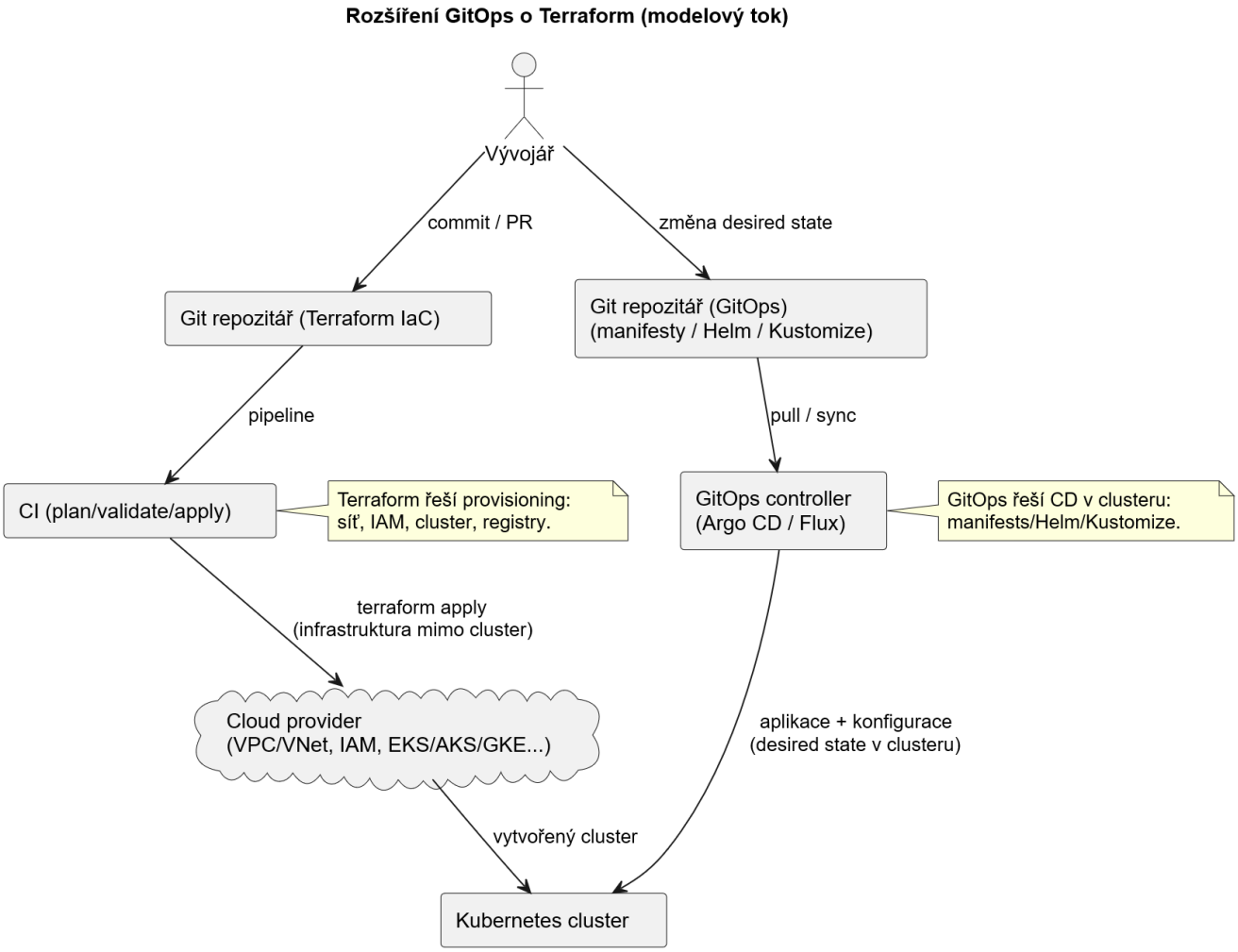
Zásadním omezením základního GitOps modelu je ukládání citlivých údajů (hesla, API klíče, certifikáty). Jelikož GitOps vyžaduje, aby byl veškerý stav deklarován v repozitáři, uložení těchto dat v prostém textu (plaintext) by představovalo kritické bezpečnostní riziko. V praxi se tento problém řeší dvěma způsoby. Prvním je šifrování tajemství přímo v Gitu pomocí nástrojů jako Sealed Secrets nebo SOPS, kde k dešifrování dochází až uvnitř clusteru. Druhým, pro enterprise prostředí vhodnějším přístupem, je integrace na externí trezory typu HashiCorp Vault nebo AWS Secrets Manager (Vinto & Soto, 2023). Git repozitář v takovém případě obsahuje pouze odkazy na tato tajemství, čímž je zachována bezpečnost i deklarativní přístup.

Role Terraformu v širším ekosystému infrastruktury

V rámci praktické části byl GitOps (Argo CD) využit k orchestraci prostředků uvnitř běžícího Kubernetes clusteru (Day-2 operations). V reálném cloudovém prostředí je však nutné vyřešit i samotné vytvoření tohoto clusteru a související infrastruktury (sítě, databáze, virtuální stroje – Day-0/Day-1 operations). Zde spočívá klíčová role nástroje Terraform. Ten je nástroj ideální pro tzv. infrastrukturu jako kód (IaC) na úrovni poskytovatele cloudu. Nejlepší praxí je proto kombinovaný přístup: Terraform zřídí cloudovou infrastrukturu a Kubernetes cluster, a jakmile je cluster dostupný, předá správu aplikační a konfigurační vrstvy GitOps operátorovi (Argo CD). Tento hybridní model pokrývá celý životní cyklus od hardwarových prostředků až po aplikační kód (Kim et al., 2016).

Vztah mezi Terraform (IaC) a GitOps (CD v Kubernetes) shrnuje Obrázek 6.6, který ukazuje doporučené rozdělení odpovědností: Terraform řeší provisioning infrastruktury mimo cluster, zatímco GitOps controller průběžně rekonsiluje desired state uvnitř clusteru.

Obrázek 6.6 – Rozšíření GitOps o Terraform (modelový tok).

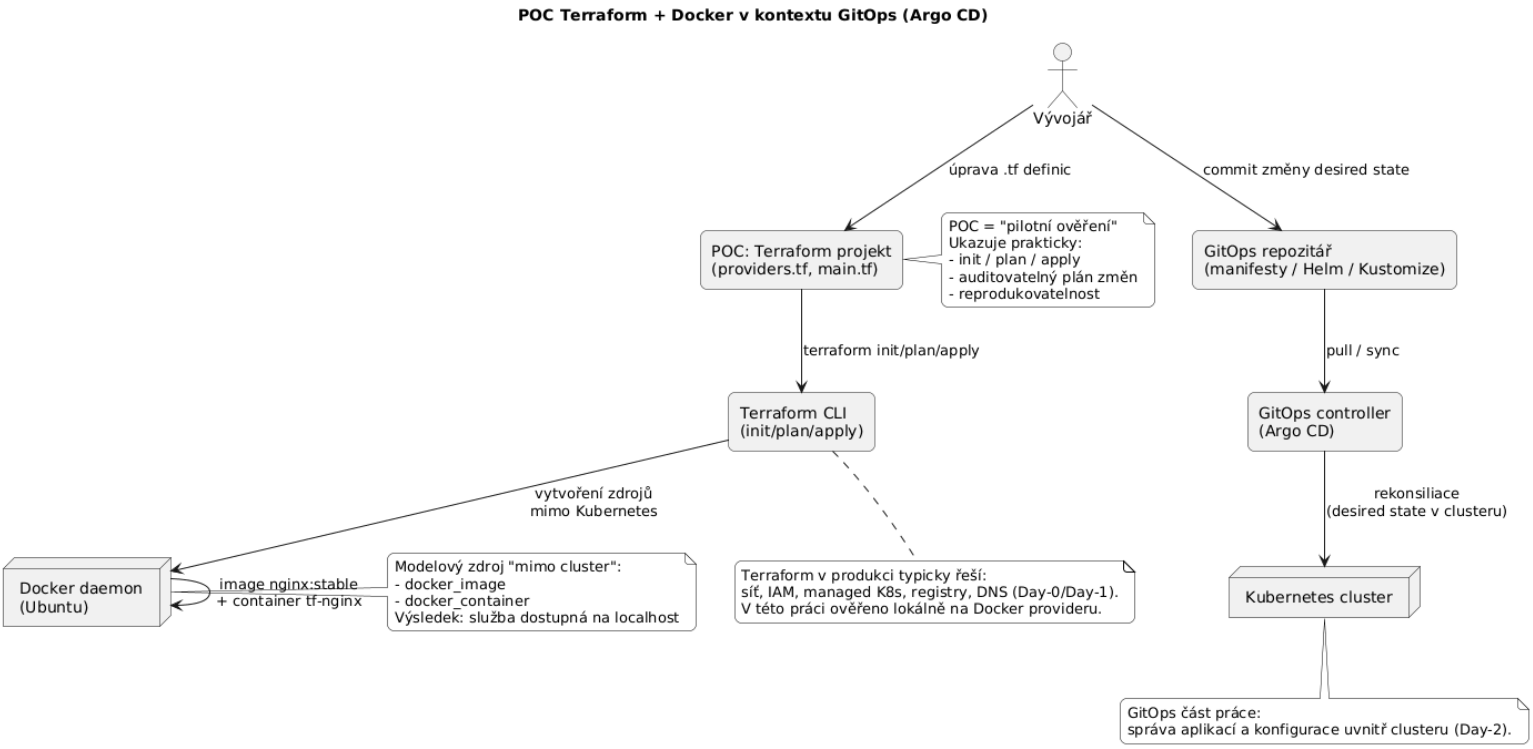


Zdroj: vlastní zpracování.

Modelový postup je v rámci testovacího prostředí navázán na prostředí Ubuntu (viz kapitola 4). Docker zde vystupuje jako podpůrná technologie pro lokální práci s kontejnery (build/test image), nikoliv jako náhrada GitOps controlleru.

Následující schéma shrnuje hlavní artefakty a části praktické realizace provedené v testovacím prostředí. Zahrnuje repozitář gitops-demo, demonstrační aplikaci nginx-demo spravovanou nástrojem Argo CD, lokální proof-of-concept Terraformu (poc/terraform-docker) a experimentální ověření vybraných principů Git internals na základě projektu WYAG.

Obrázek 6.7 – Modelový tok GitOps s využitím Argo CD a lokálního POC Terraformu



Zdroj: vlastní zpracování.

Limity a organizační omezení

Zavedení GitOps není pouze technickou, ale i procesní výzvou. Hlavním omezením pro organizace může být strmá křivka učení (steep learning curve) a nutnost přechodu na striktní verzování všeho formou pull requestů. Pokud vývojáři a operátoři nemají zažité principy DevOps a review kódu, GitOps může proces nasazování naopak zpomalit. Dále je nutné počítat s režijními nároky na provoz samotného GitOps controlleru a správu více repozitářů (např. oddělení repozitářů pro aplikační kód a pro infrastrukturní manifesty).

Očekávané přínosy rozšíření:

- vyšší auditovatelnost a opakovatelnost změn infrastruktury (IaC v Git),

- jasné oddělení odpovědností: Terraform = provisioning, GitOps = rekonsiliace v clusteru,

- snížení rizika manuálních chyb při Day-0/Day-1 operacích díky řízenému PR workflow.

Omezení:

- uvedený postup je návrh rozšíření a jeho realizace vyžaduje přístupy k cílovému cloudu a správně nastavený remote state backend.

Možné rozšíření: observability (monitoring)

V produkčním nasazení je vhodné doplnit GitOps přístup o vrstvu observability, např. pomocí Prometheus a Grafana. Tyto nástroje lze nasazovat deklarativně (např. Helm chart) a spravovat jejich konfiguraci verzovaně v Git, čímž se zachová auditovatelnost a konzistence napříč prostředími. V rámci této práce nebyla observability vrstva prakticky implementována.

6.4 Problémy během implementace a jejich řešení

Během přípravy testovacího prostředí se vyskytlo několik drobných technických překážek:

- Po spuštění příkazu microk8s enable došlo zpočátku k chybě oprávnění. Situace byla vyřešena přidáním uživatele do skupiny microk8s a následným restartem relace.
- Po přidání uživatele do skupiny bylo potřeba relaci znovu načíst, aby se oprávnění projevila. To bylo provedeno pomocí příkazu newgrp microk8s.
- Po opětovném ověření stavu klastru byly ještě jednou zaznamenány problémy s oprávněním. Tyto problémy byly vyřešeny trvalým přihlášením do nové skupiny a nastavením aliasu pro kubectl.

Tento problém z praxe jasně ukazuje úskalí manuální konfigurace infrastruktury (Day-0 operace). Pokud by byl Kubernetes cluster zřizován plně automatizovaně pomocí nástrojů jako Terraform nebo Ansible (jak je navrhováno v diskuzi), nastavení uživatelských oprávnění a přiřazení do skupin by bylo součástí definovaného kódu. Tím by se předešlo manuálním chybám a nutnosti ad-hoc řešení problémů s přístupem v příkazové řádce.

7. Závěr

Diplomová práce se zabývala problematikou implementace metodologie GitOps pro správu cloudové infrastruktury. Hlavním zaměřením práce bylo analyzovat principy tohoto přístupu, porovnat dostupné nástroje a na základě získaných poznatků popsat modelový postup jeho využití v prostředí moderní cloudové infrastruktury.

V teoretické části práce byly nejprve představeny základní koncepty související se správou infrastruktury v moderních IT systémech. Pozornost byla věnována především přístupu Infrastructure as Code (IaC), který představuje významný krok směrem k automatizaci správy infrastruktury. V této souvislosti byl popsán vývoj správy infrastruktury od tradiční manuální konfigurace serverů a systémů přes přístup Infrastructure as Code až po současný koncept GitOps. Tento vývoj ukazuje postupný přechod od manuálních a často obtížně reprodukovatelných postupů ke deklarativnímu a automatizovanému řízení infrastruktury.

Dále byly v teoretické části vysvětleny základní principy metodologie GitOps. Pozornost byla věnována především využití Git repozitáře jako centrálního zdroje pravdy pro definici požadovaného stavu infrastruktury a aplikací. Popsán byl také princip deklarativní konfigurace, automatizované synchronizace mezi repozitářem a cílovým prostředím a průběžné rekonsiliace mezi požadovaným a skutečným stavem systému. Součástí teoretické části bylo také shrnutí hlavních přínosů GitOps přístupu, mezi které patří zejména auditovatelnost změn, konzistence konfigurace a vyšší míra automatizace při správě infrastruktury.

V analytické části práce byla provedena analýza vybraných nástrojů, které se v současnosti používají při implementaci GitOps přístupu. Byly popsány nástroje Argo CD a Flux jako specializované GitOps controllery pro správu aplikací v prostředí Kubernetes. Dále byla analyzována role nástroje Terraform, který je široce využíván pro deklarativní správu infrastruktury pomocí přístupu Infrastructure as Code. Součástí analýzy byl také nástroj Helm, který slouží jako balíčkovací systém pro aplikace v Kubernetes a umožňuje jednodušší správu a verzování aplikačních konfigurací. U jednotlivých nástrojů byly popsány jejich základní principy fungování, možnosti využití a role v rámci GitOps workflow.

Na základě poznatků získaných v teoretické a analytické části byl v další části práce popsán modelový postup implementace GitOps v cloudovém prostředí. Pozornost byla věnována architektuře řešení, která kombinuje využití Git repozitáře, GitOps controlleru a orchestrace aplikací v Kubernetes clusteru. V rámci práce byl popsán proces nasazení aplikace

pomocí nástroje Argo CD a jeho integrace s GitHub repozitářem obsahujícím deklarativní definici požadovaného stavu aplikace.

Diplomová práce tak poskytuje ucelený přehled o principu GitOps, o nástrojích používaných při jeho implementaci a o způsobu jeho využití při správě moderní cloudové infrastruktury.

Seznam použitých zdrojů

ARUNDEL, John a Justin DOMINGUS. 2019. Cloud Native DevOps with Kubernetes: Building, Deploying, and Scaling Modern Applications in the Cloud. Sebastopol: O'Reilly Media, 2019. ISBN 9781492040767.

ARGO CD. 2024. Argo CD Documentation. [Online] 2024. https://argo-cd.readthedocs.io/

AWS. 2023. Amazon EKS Documentation. [Online] 2023. https://docs.aws.amazon.com/eks/

AZURE. 2024. Azure Kubernetes Service (AKS) Documentation. [Online] 2024. https://learn.microsoft.com/en-us/azure/aks/

BITNAMI. 2023. Sealed Secrets Documentation. [Online] 2023. https://github.com/bitnami-labs/sealed-secrets

BRIKMAN, Yevgeniy. 2025. Fundamentals of DevOps and Software Delivery. Sebastopol: O'Reilly Media, 2025. ISBN 9781098174590.

BUELTA, Jaime. 2019. Using GitOps Principles. In: Hands-On Docker for Microservices with Python. Birmingham: Packt Publishing, 2019. ISBN 9781838823818.

CNCF. 2023. Cloud Native Computing Foundation Landscape. [Online] 2023. https://landscape.cncf.io/

CNCF. 2021. OpenGitOps Principles. [Online] 2021. https://opengitops.dev/

DAVIS, Jennifer a Katherine DANIELS. 2016. Effective DevOps. Sebastopol: O'Reilly Media, 2016. ISBN 9781491926307.

DOCKER. 2023. Docker Documentation. [Online] 2023. https://docs.docker.com/

FLUX CD. 2024. Flux Documentation. [Online] 2024. https://fluxcd.io/docs/

FORSGREN, Nicole, Jez HUMBLE a Gene KIM. 2018. Accelerate: The Science of Lean Software and DevOps. Portland: IT Revolution Press, 2018. ISBN 9781942788331.

GITHUB. 2024. GitHub Actions Documentation. [Online] 2024. https://docs.github.com/

GOOGLE CLOUD. 2024. Google Kubernetes Engine Documentation. [Online] 2024. https://cloud.google.com/kubernetes-engine/docs

HIGHTOWER, Kelsey, Brendan BURNS a Joe BEDA. 2021. Kubernetes: Up and Running. 3rd ed. Sebastopol: O'Reilly Media, 2021. ISBN 9781098110208.

HASHICORP. 2024. Terraform Documentation. [Online] 2024. https://developer.hashicorp.com/terraform/docs

HELM. 2024. Helm Documentation. [Online] 2024. https://helm.sh/docs/

HAFNER, Robert. 2025. Terraform in Depth: Infrastructure as Code with Terraform and OpenTofu. 1. vyd. Shelter Island: Manning Publications, 2025. ISBN 9781633438002.

JOYNER, John a Steve BUCHANAN. 2021. GitOps Insights. In: Azure Arc-Enabled Kubernetes and Servers. Apress, 2021. ISBN 9781484277676.

KIM, Gene, Patrick DEBOIS, John WILLIS a Jez HUMBLE. 2016. The DevOps Handbook. Portland: IT Revolution Press, 2016. ISBN 9781942788003.

KUBERNETES. 2024. Kubernetes Documentation. [Online] 2024. https://kubernetes.io/docs/

MICROK8S. 2024. MicroK8s Documentation. [Online] 2024. https://microk8s.io/docs

MISHRA, Pravin. 2024. GitOps: The Future of Operations. In: Ultimate Git and GitHub for Modern Software Development. Orange Education, 2024.

MORRIS, Kief. 2021. Infrastructure as Code: Dynamic Systems for the Cloud Age. 2nd ed. Sebastopol: O'Reilly Media, 2021. ISBN 9781492092604.

OPA. 2023. OPA Gatekeeper Documentation. [Online] 2023. https://open-policy-agent.github.io/gatekeeper/

ORACLE. 2023. VirtualBox User Manual. [Online] 2023. https://www.virtualbox.org/manual/

POLGE, Thibault. Write yourself a Git! [online]. 2023 . https://wyag.thb.lt/

RED HAT. 2022. What is GitOps? [Online] 2022. https://www.redhat.com/en/topics/devops/what-is-gitops

SALECHA, Rohit. 2022. Practical GitOps: Infrastructure Management Using Terraform, AWS, and GitHub Actions. Berkeley: Apress, 2022. ISBN 9781484286739.

SPACELIFT. 2023. Terraform and GitOps Integration. [Online] 2023. https://spacelift.io/blog

STACK OVERFLOW. 2023. Kubernetes Drift Detection. [Online] 2023. https://stackoverflow.com/

TEKTON. 2023. Tekton Documentation. [Online] 2023. https://tekton.dev/docs/

TERRAFORM REGISTRY. 2024. Terraform Providers Documentation. [Online] 2024. https://registry.terraform.io/

VAN DER VLIST, Eric. 2021. Continuous Delivery in Kubernetes. O'Reilly, 2021.

VANSTONE, Scott. 2020. Infrastructure Automation Patterns. Apress, 2020.

VINTO, Natale a Alex SOTO BUENO. 2023. GitOps Cookbook: Kubernetes Automation in Practice. Sebastopol: O'Reilly Media, 2023. ISBN 9781492097471.

WEAVEWORKS. 2021. GitOps Whitepaper. [Online] 2021. https://www.weave.works/technologies/gitops/

YOUNG, Alex. 2023. Kubernetes Security Best Practices. [Online] 2023. https://kubernetes.io/blog/

YUEN, David et al. 2021. Argo CD: Up and Running. O'Reilly Media, 2021.

Seznam obrázků, tabulek, grafů a zkratek

- Obrázek 2.1 – CI/CD-pipeline
- Obrázek 2.2 – Argo CD – architektura a smyčka synchronizace
- Obrázek 2.3 – Model nasazení aplikace (push vs. pull)
- Obrázek 2.4 – GitOps loop
- Obrázek 2.5 – Správa tajemství pomocí Sealed Secrets v GitOps
- Obrázek 3.1 – Schematické znázornění GitOps workflow
- Obrázek 3.2 – Integrace Terraform a GitOps (Argo CD/Flux) v nasazovacím procesu
- Obrázek 4.1 – Architektura GitOps nasazení (GitHub → Argo CD → Kubernetes)
- Obrázek 4.2 – Instalace nástrojů Git, Docker a curl v systému Ubuntu
- Obrázek 4.3 – Stav aplikace v Argo CD po inicializaci (Healthy, Synced)
- Obrázek 4.4 – Přehled aplikací v Argo CD po vytvoření nginx-demo
- Obrázek 4.5 – Stav aplikace nginx-demo po synchronizaci v Argo CD
- Obrázek 5.1 – STEP 1: Rozhodovací model vyhodnocení výchozího stavu GitOps ve firmě
- Obrázek 5.2 – STEP 2: Technologický a procesní rámec GitOps metodiky
- Obrázek 5.3 – STEP 3A: Git jako stavový model (WYAG)
- Obrázek 5.4 – STEP 3B: GitOps Reconciliation Loop
- Obrázek 5.5 – STEP 3C: Metodické propojení WYAG a GitOps
- Obrázek 6.1 – Stav aplikace nginx-demo v Argo CD (Healthy, Synced) a strom nasazených objektů
- Obrázek 6.2 – Git repozitář jako jediný zdroj pravdy – Helm chart aplikace nginx
- Obrázek 6.3 – Ověření změny v Kubernetes: dvě repliky nginx po synchronizaci
- Obrázek 6.4 – Stav aplikace nginx-demo po synchronizaci a strom prostředků
- Obrázek 6.5 – Demonstrace self-healing procesu v příkazové řádce: simulace driftu a automatická obnova

Seznam tabulek

- Tabulka 2.1 – Přehled cloudových platforem a podpory GitOps nástrojů
- Tabulka 3.1 – Srovnání nástrojů Argo CD, Flux, Terraform a Helm z hlediska jejich role v GitOps
- Tabulka 4.1 – Testovací prostředí a použité verze

Seznam výpisů (Listings)

- Výpis 3.1 – Ukázka Terraform konfigurace
- Výpis 3.2 – Ukázka šablony Helm chartu
- Výpis 4.1 – Struktura repozitáře gitops-demo
- Výpis 4.2 – Původní desired state v repozitáři (nginx-deployment.yaml – výřez)
- Výpis 4.3 – Aktualizovaný desired state v repozitáři (nginx-deployment.yaml – výřez)
- Výpis 5.1 – Výstup příkazů Git demonstrující stavový model

Seznam zkratek

- **API** – Application Programming Interface
- **CD** – Continuous Delivery / Continuous Deployment
- **CI** – Continuous Integration
- **CLI** – Command Line Interface
- **CRD** – Custom Resource Definition
- **GUI** – Graphical User Interface
- **HCL** – HashiCorp Configuration Language
- **IaC** – Infrastructure as Code
- **RBAC** – Role-Based Access Control
- **SSO** – Single Sign-On
- **UI** – User Interface

- **VM** – Virtual Machine
- **YAML** – YAML Ain't Markup Language